

НОВОСИБИРСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ВЫСШИЙ КОЛЛЕДЖ ИНФОРМАТИКИ

Л.К. Попов

Обучение программированию с помощью языка C#

Книга 2. Windows Forms приложения

НОВОСИБИРСК
2021

Оглавление

Введение	3
2. Приложения Windows Forms	3
2.1 Начало работы	3
2.2 Элементы управления формы	7
2.2.1 Элемент Label	8
2.2.2 Элемент Button (Кнопка)	9
2.2.3 Элемент TextBox.....	10
2.2.4 Первый проект – калькулятор	12
2.3 Графика в Window Forms.....	16
2.3.1 Графические поверхности	16
2.3.2. Инструменты рисования	18
2.3.2.1. Класс Pen – карандаш.....	18
2.3.2.2. Кисть – класс Brush.	18
2.3.3. Графические примитивы.....	19
2.3.4. Работа с изображениями	21
2.4. Движение графических объектов.....	21
2.4.1. Компонент Timer	21
2.4.2. Компонент NumericUpDown	21
2.4.3. Неуправляемое движение графического объекта	22
2.4.4. Движение объекта с помощью клавиш	23
2.4.5. Движение объекта с помощью мыши.....	24
2.4.6. Светофор	25
2.4.7 Крестики-нолики	27
2.5. Многооконные программы	29
2.6. Контейнеры в Window Forms.	30
2.6.1. Контейнер GroupBox	30
2.6.2. Контейнер Panel.....	31
2.7. Переключатели в Windows Forms	31
2.7.1. Переключатель CheckBox.....	31
2.7.2. Переключатель RadioButton	33
2.8. Окно сообщений MessageBox.	34
2.9. Диалоги открытия и сохранения файлов.....	37
2.10. Списки ListBox и ComboBox.....	38
2.10.2. Список ComboBox.	42
2.11. Главное меню. Элемент MenuStrip.	43
2.12. Учебная викторина Миллионер	44
2.13. Создание компонентов программным способом	47

2.14. Компонент PictureBox.....	51
Заключение.....	54
Литература	54

Введение

Предлагаемое методическое пособие разработано для обеспечения учебного процесса дисциплины «Прикладная информатика» и предназначено для студентов и преподавателей первого курса ВКИ НГУ.

Во втором семестре обучения у этого курса двойная цель. Во-первых, обучение студентов технологии создания GUI.

Графический интерфейс пользователя ([англ. graphical user interface, GUI](#)) – это система средств для взаимодействия пользователя с компьютером, основанная на представлении всех доступных пользователю системных объектов и функций в виде графических компонентов экрана (окон, значков, меню, кнопок, списков и т. п.). Все основные операционные системы, такие как Windows, Mac, iOS и Android, имеют графический интерфейс, в котором вы можете щелкнуть значок, чтобы выполнить такое действие, как открытие приложения, просмотр меню или перемещение по вашему устройству. Известная вам оболочка Shell так же является примером GUI.

Главным преимуществом GUI является то, что эти системы, доступны для людей всех уровней знаний, от абсолютного новичка до продвинутого разработчика или других технологов. Они позволяют простым пользователям открывать меню, перемещать файлы, запускать программы или искать в Интернете, не указывая компьютеру функции для выполнения через командную строку. GUI также обеспечивают мгновенную обратную связь. Например, щелчок на значке откроет некоторое приложение, и это можно увидеть в режиме реального времени.

В настоящее время почти все приложения, которые создаются для конечного пользователя, имеют GUI.

Во-вторых, как и в первом семестре обучения, мы продолжаем обучать студентов основам алгоритмизации и, в частности, навыкам работы с двумерными массивами. Поэтому конечной целью обучения студента является курсовой проект, содержанием которого является создание логической игры, управляемой с помощью разработанного им GUI.

До недавнего времени эти задачи решались с помощью среды визуального программирования Delphi, построенной на основе языка Pascal. Предполагается в дальнейшем использовать для этих целей язык C# и среду Microsoft Visual Studio. Данное методическое пособие призвано обучить студентов работе с этими инструментами.

2. Приложения Windows Forms

2.1 Начало работы

Для работы в C# мы будем использовать среду Microsoft Visual Studio. Этот пакет установлен в сети ВКИ. Его установка на домашний компьютер не представляет трудности, и мы не будем на этом останавливаться. Мы будем использовать Microsoft Visual Studio 2019. Ее первый запуск можно осуществить нажатием кнопки Microsoft Visual Studio в

меню Пуск. Порядок начала работы с более ранними версиями можно найти, например, в [1]. В дальнейшем можно использовать пункты меню *Файл | Создать | Проект*. После запуска откроется следующее окно (рис.1.1):

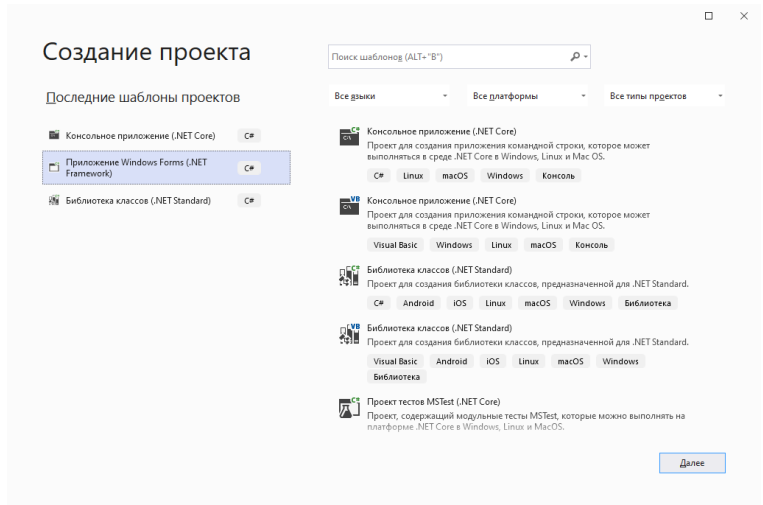


Рис.2.1 Окно проекта в Microsoft Visual Studio

Выбираем в списке проектов слева «Приложение Windows Forms» и нажимаем кнопку «Далее».

Windows-приложение создается в виде проекта. Проект состоит из нескольких файлов разного назначения. В табл. 1 приводится описание некоторых файлов проекта:

Таблица 1

Имя файла	Описание
*.sln	файл сборки проекта
*.suo	файл опций проекта
*.csproj	Проект
*.cs	программные файлы на C#
*.Designer.cs	содержит графическое представление формы

Вся работа с приложениями Windows Forms выполняется через меню «*Файл*», а именно создание, открытие, сохранение.

Имя проекта вы выбираете самостоятельно. Для сохранения проекта необходимо создать отдельную папку. Нажав на кнопку с точками и воспользовавшись открывающимся при этом диалоговым окном, вы обеспечите сохранение проекта в этой папке. Необходимо также поставить галочку «Поместить решение и проект в одном проекте». Проект находится внутри *решения*. Несмотря на название, под решением не подразумевается "решение вопроса". Это просто контейнер для одного или нескольких связанных проектов вместе с информацией о сборке, параметрами окна Visual Studio и любыми прочими файлами, которые не относятся к какому-либо конкретному проекту. Решение описывается текстовым файлом (*SLN*) в собственном уникальном формате; его обычно не изменяют вручную. Нажимаем кнопку «Создать». Откроется окно редактора Microsoft Visual Studio, в котором вы увидите свою первую форму – будущее окно Windows-приложения (рис. 2.2).

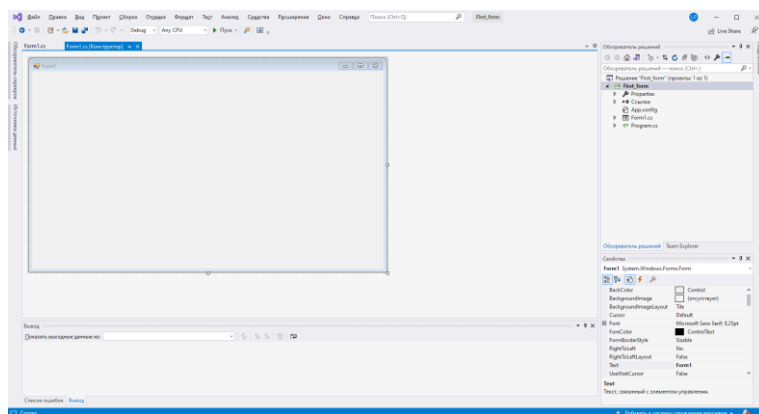


Рис. 2.2. Окно редактора Microsoft Visual Studio

Затем необходимо вывести на экран окно с кодом программы, с помощью которой вы будете работать с формой и ее компонентами. Для этого следует нажать F7 или выбрать в пункте меню «Вид» подпункт «Код». В дальнейшем не раз придется между конструктором и кодом. Для этого ниже основного меню есть специальное подменю. В итоге откроется следующий код:

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;

namespace First_form
{
    public partial class Form1 : Form
    {
        public Form1()
        {
            InitializeComponent();
        }
    }
}
```

Команды, использующие слово using, подключают к программе необходимые для ее работы пространства имен. Далее объявляется класс Form1 как наследник класса Form. Ключевое слово «partial» позволяет определению класса, структуры или интерфейса быть распределенным по нескольким файлам. Метод *InitializeComponent* позволяет инициализировать компоненты (элементы управления), которые будут помещены в форму.

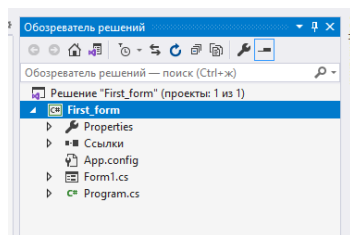


Рис. 2.3. Обзорщик решений

Вернемся в конструктор формы. В правом верхнем углу окна проекта мы видим обозреватель решений (рис. 2.3). В нем можно видеть все формы, входящие в сборку(решение), просматривать их, переключаться между ними.

В правом нижнем углу находится окно свойств и методов формы. Там же будут отображаться свойства и методы компонента (кнопки, надписи и т.д.), с которым вы в настоящий момент работаете (рис. 2.4).

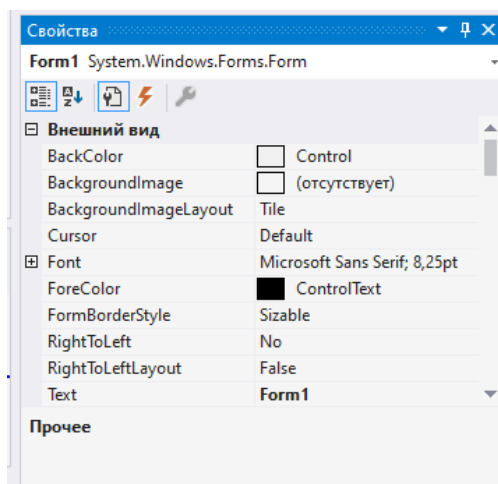


Рис. 2.4. Окно свойств и методов

Выполним нашу первую программу. Для этого нужно нажать F5 или нажать мышью на зеленую стрелку «Пуск». И мы увидим форму (окно) – наше первое Windows приложение (рис. 2.5).

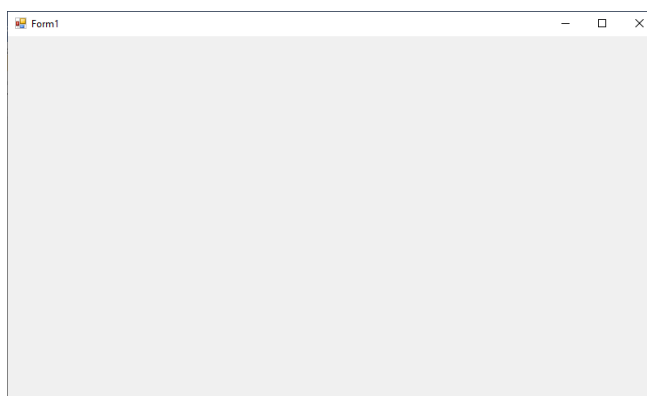


Рис. 2.5. Первое Windows приложение

Рассмотрим основные свойства формы (см. табл. 2)

Таблица 2

Свойство	Описание
Name	имя формы, как объекта класса, позволяет ссылаться на форму в программе
Text	задает заголовок окна формы
BackColor	задает цвет фона
StartPosition	задает стартовую позицию окна
Size	задает размер окна

С помощью окна свойств зададим заголовок и цвет фона нашего окна. Результат можно видеть на рис. 2.6.

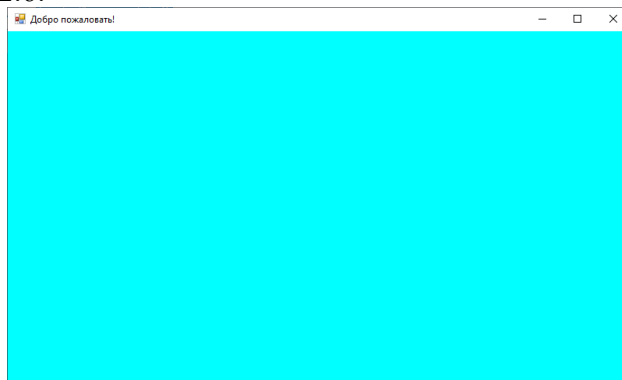


Рис. 2.6. Окно с измененными свойствами

К настоящему моменту в нашем окне ничего нет. Пришло время познакомиться с элементами управления формы. Другие их название – виджеты или компоненты формы.

Более подробно с началом работы в Windows Forms можно познакомиться в документации Microsoft [2].

2.2 Элементы управления формы

В данном разделе мы познакомимся с основными элементами управления форм: кнопками, флажками, меню, списками и т.д. Элементы управления формы могут быть как визуальными, т.е. видимыми, так и не визуальными, т.е. видимыми только на этапе проектирования приложения.

Все элементы управления располагаются на Панели элементов. Для ее вызова нужно выбрать в меню «Вид» пункт «Панель управления». На рис 2.7 показана активированная панель управления.

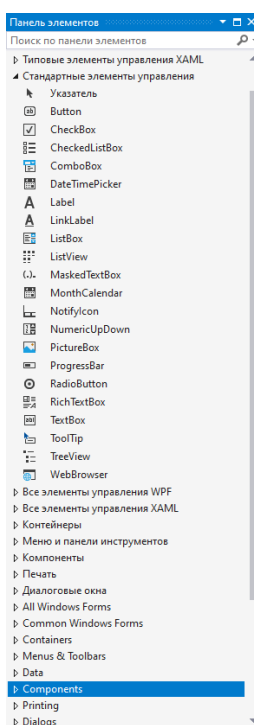


Рис. 2.7. Панель управления

Панель элементов состоит из нескольких вкладок:

- **Стандартные элементы управления** – содержит основные элементы управления форм, такие как метки, списки, кнопки и т.д.
- **Контейнеры** – элементы контейнеры.
- **Меню и панели инструментов** – меню и кнопочные панели.
- **Данные** – элементы для работы с данными (базами данных).
- **Компоненты** – различные вспомогательные компоненты, такие как Timer, ImageList, Process, DirectorySearcher.
- **Печать** – элементы для печати.
- **Диалоговые окна** – для создания стандартных диалоговых окон Windows.
- **Отчеты** – компоненты для создания отчетов.

Все элементы управления имеют ряд общих свойств:

- **Anchor:** Определяет, как элемент будет растягиваться
- **BackColor:** Определяет фоновый цвет элемента
- **BackgroundImage:** Определяет фоновое изображение элемента
- **ContextMenu:** Контекстное меню, которое открывается при нажатии на элемент правой кнопкой мыши. Задается с помощью элемента ContextMenu
- **Cursor:** Представляет, как будет отображаться курсор мыши при наведении на элемент
- **Dock:** Задаёт расположение элемента на форме
- **Enabled:** Определяет, будет ли доступен элемент для использования. Если это свойство имеет значение False, то элемент блокируется.
- **Font:** Устанавливает шрифт текста для элемента
- **ForeColor:** Определяет цвет шрифта
- **Location:** Определяет координаты верхнего левого угла элемента управления
- **Name:** Имя элемента управления
- **Size:** Определяет размер элемента
- **Width:** ширина элемента
- **Height:** высота элемента
- **TabIndex:** Определяет порядок обхода элемента по нажатию на клавишу Tab
- **Tag:** Позволяет сохранять значение, ассоциированное с этим элементом управления

Для того, чтобы разместить элемент на форме необходимо перетащить его мышью с панели управления на форму.

2.2.1 Элемент Label

Для отображения простого текста на форме, доступного только для чтения, служит элемент Label (метка). Основные его свойства приведены в табл. 3.

Таблица 3

Свойство	Описание
Name	Задаёт имя объекта, используется для ссылки на объект в программе

Text	Строка, задает тот текст, который будет выведен в окне
Autosize	Если свойство равно true, то рамка объекта растягивается по размеру текста.
Font	Задаёт шрифт (размер, стиль, подчеркивание, зачеркивание)
Image	Задаёт рисунок, связанный с текстом
TextAlign	Задаёт выравнивание текста относительно рамки
BackColor	Задаёт цвет фона

Разместим на нашей форме Label и с помощью свойства Текст создадим на ней надпись. Результат можно видеть на рис. 2.8.

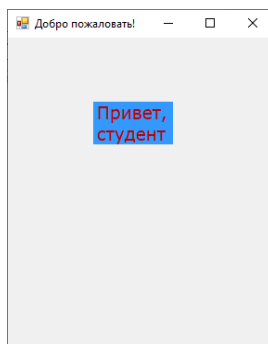


Рис. 2.8. Элемент Label.

2.2.2 Элемент Button (Кнопка)

Наиболее часто используемым элементом управления является кнопка. Обработывая событие нажатия кнопки, мы можем производить те или иные действия.

Основные свойства кнопки приведены в табл. 4:

Таблица 4

Свойство	Описание
Text	Текст на кнопке
TextAlign	Выравнивание текста относительно рамки
Size	Размер кнопки
Font	Шрифт
ForeColor	Цвет букв
BackColor	Цвет фона
Image	Картинка на кнопке
ImageAlign	Выравнивание картинки относительно рамки

Установим для кнопки изображение. Для этого выберем кнопку и в окне Свойств нажмем на поле Image. Нам откроется диалоговое окно установки изображения(рис.2.9):

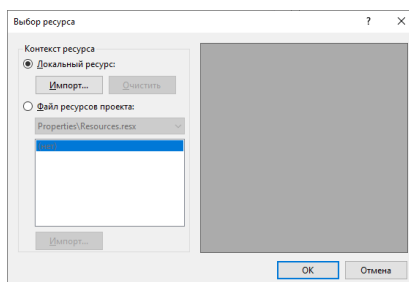


Рис.2.9. Окно установки изображения

В этом окне выберем опцию Local Resource и нажмем на кнопку Import, после чего нам откроется диалоговое окно для выбора файла изображения. После выбора изображения мы можем установить свойство **ImageAlign**, которое управляет позиционированием изображения на кнопке. Нам доступны 9 вариантов, с помощью которых мы можем прикрепить изображение к определенной стороне кнопки. Оставим здесь значение по умолчанию - MiddleCenter, то есть позиционирование по центру. Затем перейдем к свойству TextImageRelation и установим для него значение ImageBeforeText. В итоге мы получим кнопку, где сразу после изображения идет надпись на кнопке.

При нажатии на кнопку в редакторе проекта мы получаем в коде программы обработчик события. В нем мы описываем действия, которые программа должна будет выполнить при нажатии кнопки после запуска программы на исполнение.

```
public partial class Form1 : Form
{
    public Form1()
    {
        InitializeComponent();
    }

    private void button1_Click(object sender, EventArgs e)
    {
        label1.Text += "! Где твой мяч?";
    }
}
```

Результаты работы программы до (слева) и после нажатия кнопки представлены на рис. 2.10.

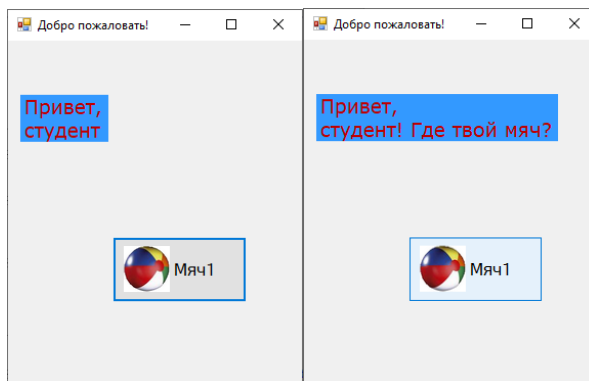


Рис.2.10 Результаты работы кнопки

2.2.3 Элемент TextBox

Для ввода и редактирования текста предназначены текстовые поля - элемент TextBox. Также как и у элемента Label текст элемента TextBox можно установить или получить с помощью свойства Text.

Основные свойства элемента приведены в табл. 5:

Таблица 5

Свойство	Описание
Text	Строка, содержит то, что ввел пользователь
TextAlign	Задаёт выравнивание текста относительно рамки
Lines	Свойство типа массив, содержит многострочный текст, который введен в поле.
MultiLine	Если свойство равно true, то элемент будет показывать многострочный текст
Location	Задаёт расположение элемента относительно окна
Font	Шрифт текста
BackColor	Цвет фона
ForeColor	Цвет текста
WordWrap	Если свойство равно true, то введенный текст разбивается на несколько строк по пробелам
PasswordChar	Задаёт символ, маскирующий ввод
ReadOnly	Если равен true, элемент только для чтения
ScrollBars	Позволяет добавить полосу скроллинга, если равен true

По умолчанию при переносе элемента с панели инструментов создается однострочное текстовое поле. Для отображения больших объемов информации в текстовом поле нужно использовать его свойства Multiline и ScrollBars. При установке для свойства Multiline значения true, все избыточные символы, которые выходят за границы поля, будут переноситься на новую строку.

Кроме того, можно сделать прокрутку текстового поля, установив для его свойства ScrollBars одно из значений:

- **None:** без прокруток (по умолчанию)
- **Horizontal:** создает горизонтальную прокрутку при длине строки, превышающей ширину текстового поля
- **Vertical:** создает вертикальную прокрутку, если строки не помещаются в текстовом поле
- **Both:** создает вертикальную и горизонтальную прокрутку

Пример формы с текстовым полем представлен на рис. 2.11

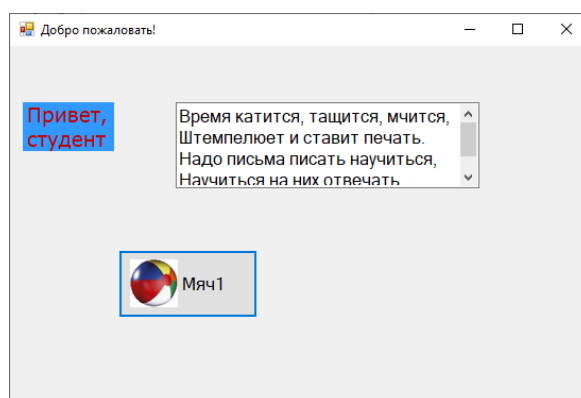


Рис. 2.11. Окно с текстовым полем

2.2.4 Первый проект – калькулятор

Калькулятор содержит взаимодействующие между собой кнопки и TextBox. Разработка такой программы полезна для обучения работе с этими элементами управления. Ниже мы приводим код программы и ее результат – окно калькулятора (рис. 2.12).

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;

namespace Calculator
{
    public partial class Form1 : Form
    {
        public Form1()
        {
            InitializeComponent();
        }
        int nop; //Номер операции
        double a, b; //Числа для операций
        private void button1_Click(object sender, EventArgs e)
        {
            if (textBox1.Text=="0") // Если 0 - заменяем на 1
                textBox1.Text = button1.Text;
            else
                textBox1.Text += button1.Text; //Иначе добавляем 1
        }

        private void button2_Click(object sender, EventArgs e)
        {
            if (textBox1.Text == "0")//Все цифры вводятся одинаково
                textBox1.Text = button2.Text;
            else
                textBox1.Text += button2.Text;
        }

        private void button3_Click(object sender, EventArgs e)
        {
            if (textBox1.Text == "0")
                textBox1.Text = button3.Text;
            else
                textBox1.Text += button3.Text;
        }

        private void button4_Click(object sender, EventArgs e)
        {
            if (textBox1.Text == "0")
                textBox1.Text = button4.Text;
            else
                textBox1.Text += button4.Text;
        }

        private void button5_Click(object sender, EventArgs e)
        {

```

```

        if (textBox1.Text == "0")
            textBox1.Text = button5.Text;
        else
            textBox1.Text += button5.Text;
    }

    private void button6_Click(object sender, EventArgs e)
    {
        if (textBox1.Text == "0")
            textBox1.Text = button6.Text;
        else
            textBox1.Text += button6.Text;
    }

    private void button7_Click(object sender, EventArgs e)
    {
        if (textBox1.Text == "0")
            textBox1.Text = button7.Text;
        else
            textBox1.Text += button7.Text;
    }

    private void button8_Click(object sender, EventArgs e)
    {
        if (textBox1.Text == "0")
            textBox1.Text = button8.Text;
        else
            textBox1.Text += button8.Text;
    }

    private void button9_Click(object sender, EventArgs e)
    {
        if (textBox1.Text == "0")
            textBox1.Text = button9.Text;
        else
            textBox1.Text += button9.Text;
    }

    private void button10_Click(object sender, EventArgs e)
    {
        if (!(textBox1.Text == "0")) // Если кроме 0 ничего нет
            textBox1.Text += button10.Text; // Второй ноль не ставим
    }

    private void button15_Click(object sender, EventArgs e)
    {
        //Кнопка +
        try //Проверка ошибки ввода
        {
            a = Convert.ToDouble(textBox1.Text);
            textBox1.Text = "0";
            nop = 1; //Операциям присваиваем номер
        }
        catch (Exception ex)
        {
            MessageBox.Show("Это не число", "Ошибка");
            textBox1.Text = "0";
        }
    }

    private void button13_Click(object sender, EventArgs e)
    {
        textBox1.Text = "0"; //Стирание
    }

```

```

private void button12_Click(object sender, EventArgs e)
{
    string s = textBox1.Text;

    if (s[0] == '-')//Если перед числом минус - удаляем
    {
        s = s.Remove(0, 1);
        textBox1.Text = s;
    }
    else // Если минуса нет - ставим
    if (!((textBox1.Text == "0") || (textBox1.Text == "0,")))
    {
        s="-"+s;
        textBox1.Text = s;
    }
}

private void button16_Click(object sender, EventArgs e)
{
    try
    {
        a = Convert.ToDouble(textBox1.Text);
        textBox1.Text = "0";
        nop = 2;
    }
    catch (Exception ex)
    {
        MessageBox.Show("Это не число", "Ошибка");
        textBox1.Text = "0";
    }
}

private void button17_Click(object sender, EventArgs e)
{
    try
    {
        a = Convert.ToDouble(textBox1.Text);
        textBox1.Text = "0";
        nop = 3;
    }
    catch (Exception ex)
    {
        MessageBox.Show("Это не число", "Ошибка");
        textBox1.Text = "0";
    }
}

private void button18_Click(object sender, EventArgs e)
{
    try
    {
        a = Convert.ToDouble(textBox1.Text);
        textBox1.Text = "0";
        nop = 4;
    }
    catch (Exception ex)
    {
        MessageBox.Show("Это не число", "Ошибка");
        textBox1.Text = "0";
    }
}

```

```

private void button14_Click(object sender, EventArgs e)
{
    bool go = true;
    try
    { // Считываем второе число
        b = Convert.ToDouble(textBox1.Text);
    }
    catch (Exception ex)
    {
        MessageBox.Show("Это не число", "Ошибка");
        textBox1.Text = "0";
    }
    switch(nop)
    {
        case 1:
            a = a + b;
            break;
        case 2:
            a = a - b;
            break;
        case 3:
            a = a * b;
            break;
        case 4:
            {
                if (b == 0)
                {
                    textBox1.Text = "0";
                    go = false;
                    MessageBox.Show("Деление на 0", "Ошибка");
                }
                else a = a / b;
            }
            break;
    }
    if (go) textBox1.Text = a.ToString();
}

private void button11_Click(object sender, EventArgs e)
{ //Если запятой нет - ставим
    if (textBox1.Text.IndexOf(",") == -1)
    {
        textBox1.Text += ",";
    }
}

}
}
}

```

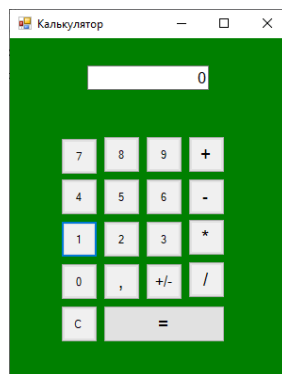


Рис. 2.12. Калькулятор

2.3 Графика в Window Forms

2.3.1 Графические поверхности

Перед тем как начать рисовать линии, фигуры, выводить текст и манипулировать другими графическими объектами, необходимо создать графическую поверхность. Это правило действует во всех языках программирования. В библиотеке `pygame` языка Python – это `Surface`, в библиотеке того же языка `tkinter` – `Canvas` (холст), в среде Delphi – `Canvas` и т. д. В C# эта поверхность называется `Graphics`. Создать ее можно следующими способами:

Способ 1.

Обработчик события `Paint` формы в качестве параметра передает объект `e` типа `PaintEventArgs`. Приведем пример обработчика:

```
private void Form1_Paint(object sender, PaintEventArgs e)
{
    // Создание графической поверхности
    Graphics g = e.Graphics;
    // Код для рисования вставляется сюда
}
```

Способ 2.

С помощью класса `Image`. Для создания графической поверхности из объекта типа `Image` вызывается метод `Graphics.FromImage`.

```
// создается объект типа Bitmap
Bitmap myBitmap = new Bitmap(@"C:\temp\myPic.bmp");
Graphics g = Graphics.FromImage(myBitmap);
```

Способ 3.

Используется метод `CreateGraphics` элемента управления или формы для получения ссылки на графический объект, который позволяет рисовать в форме или по элементу управления.

```
g = this.CreateGraphics();
```

где `this` – это ссылка на форму.

В ближайшем будущем мы будем использовать третий способ. Приведем пример. Создадим поверхность и нарисуем на ней линию и прямоугольник. Подробно технику создания линии и других графических объектов мы рассмотрим ниже. Сейчас сосредоточимся на создании графической поверхности. Алгоритм работы программы будет понятен из комментариев к ней.

Пример 2.3.1. Создание поверхности и на ней – линии и прямоугольника.

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
```



```

using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;

namespace My_Line
{
    public partial class Form1 : Form
    {
        public static int y = 10;
        Graphics g; //Объявляем переменную g типа Graphics
        public Form1()
        {
            InitializeComponent();
            g = this.CreateGraphics(); //Создаем поверхность на форме
        }

        private void Form1_Paint(object sender, PaintEventArgs e)
        {
            //Рисуем линию с помощью Form1_Paint
            Pen myPenGreen = new Pen(Color.Green, 3);
            g.DrawLine(myPenGreen, 10, y, 100, 100);
        }

        private void button1_Click(object sender, EventArgs e)
        {
            //Нажимаем на кнопку и рисуем прямоугольник
            Pen myPenGreen = new Pen(Color.Green, 3);
            g.DrawRectangle(myPenGreen, y, 100, 100, 100);
        }
    }
}

```

В результате получается следующее окно.

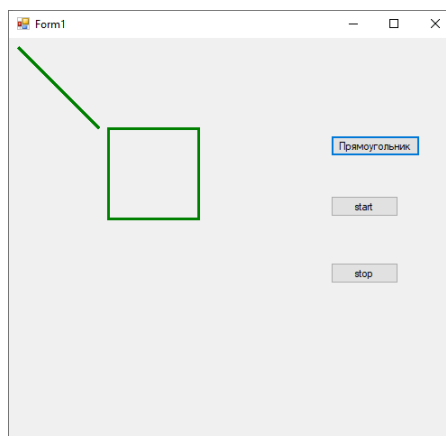


Рис. 2.13. Линия и прямоугольник

У класса Graphics много различных методов. Обратим внимание на три из них (табл. 6).

Таблица 6.

Clear(Color)	Очищает всю поверхность рисования и выполняет заливку поверхности указанным цветом фона.
--------------	--

CopyFromScreen(Point, Point, Size)	Выполняет передачу данных о цвете, соответствующих прямоугольной области пикселей, блоками битов с экрана на поверхность рисования объекта Graphics.
Dispose()	Освобождает все ресурсы, используемые данным объектом Graphics.

2.3.2. Инструменты рисования

После создания графической поверхности можно рисовать линии, фигуры, поверхности. Для этого используются следующие классы.

- Класс **Pen** - используется для рисования линий, поверхностей или других геометрических представлений.
- Класс **Brush** – используется для заполнения фигур и изображений цветом.
- Класс **Font** – обеспечивает текст нужным шрифтом, при этом задается тип шрифта, размер шрифта и т.д.
- Структура **Color** -представляет различные цвета.

Перед рисованием фигур и линий необходимо задать карандаш и кисть. Карандаш задает цвет и толщину линии, кисть задает цвет и тип заливки.

2.3.2.1. Класс Pen – карандаш.

Объект карандаш можно создать следующим образом:

```
Pen myPenRed = new Pen();
```

Карандаш имеет следующие основные свойства:

Color – задает цвет карандаша. Например:

```
Color.Red // задается красный цвет.
```

Width – задает толщину карандаша. Например:

```
myPenRed.Width = 3;
```

Можно создать карандаш следующим образом: задавая два параметра, первый – цвет, второй – толщина:

```
Pen myPenRed = new Pen(Color.Red,3);
```

2.3.2.2. Кисть – класс Brush.

Brush - это абстрактный класс, от которого происходят различные классы, задающие разные типы кистей (табл. 7).

Таблица 7

Класс	Тип кисти
SolidBrush	задает кисть сплошной заливки

TextureBrush	задает текстурную кисть, закрашивает область растровым изображением
LinearGradientBrush	задает градиентную кисть, обеспечивает плавное смешение двух цветов для получения градиентного перехода.

Пример:

```
SolidBrush myBrush1 = new SolidBrush(Color.White);
```

В дальнейшем мы в основном будем использовать класс SolidBrush. Порядок работы с двумя другими классами можно найти в [].

Рассмотрим теперь коды программ, рисующих графические примитивы.

2.3.3. Графические примитивы.

Любая картинка, чертеж, схема представляют собой совокупность графических *примитивов*: точек, линий, окружностей, дуг, текста и др.

Вычерчивание графических примитивов на графической поверхности (Graphics) выполняют соответствующие методы (табл. 7).

Таблица 7.

Метод	Действие	Пример
DrawLine(Pen, x1, y1, x2, y2)	Рисует линию. Параметр Pen определяет цвет, толщину и стиль линии; параметры x1, y1, x2, y2 — координаты точек начала и конца линии	Pen myPenGreen = new Pen(Color.Green, 3); g.DrawLine(myPenGreen, 10, 10, 350, 350);
DrawRectangle(Pen, x, y, w, h)	Рисует контур прямоугольника. Параметр Pen определяет цвет, толщину и стиль границы прямоугольника; параметры x, y — координаты левого верхнего угла; параметры w и h задают размер прямоугольника	Pen myPenRed = new Pen(Color.Red, 3); g.DrawRectangle(myPenRed, 15, 15, 250, 250);
FillRectangle(Brush, x, y, w, h)	Рисует закрашенный прямоугольник. Параметр Brush определяет цвет и стиль закрашки прямоугольника; параметры x, y — координаты левого верхнего угла; параметры w и h задают размер прямоугольника	<code>SolidBrush myBrush1 = new SolidBrush(Color.White);</code> g.FillRectangle(myBrush1, 10, 10, 100, 40);
DrawEllipse(Pen, x, y, w, h)	Рисует эллипс (контур). Параметр Pen определяет цвет, толщину и стиль линии эллипса; параметры x, y, w, h — координаты левого верхнего угла и размер прямоугольника, внутри которого вычерчивается эллипс	Pen blackPen = new Pen(Color.Black); g.DrawEllipse(blackPen, 10, 10, 100, 100);
FillEllipse(Brush, x, y, w, h)	Рисует закрашенный эллипс. Параметр Brush определяет цвет и стиль закрашки внутренней области эллипса; параметры x, y, w, h — координаты левого верхнего угла и размер прямоугольника, внутри которого вычерчивается эллипс	<code>SolidBrush RedBr = new SolidBrush(Color.Red);</code> g.FillEllipse(RedBr, 10, 10, 100, 100);
DrawPolygon(Pen, P)	Рисует контур многоугольника. Параметр Pen определяет цвет, толщину и стиль линии границы многоугольника; параметр P (массив типа Point) — координаты углов многоугольника	g.DrawPolygon(myPen, new PointF[] {new PointF(1, 1), new PointF(20, 10), new PointF(100, 2), new PointF(200, 50)});

FillPolygon(Brush, P)	Рисует закрашенный многоугольник. Параметр Brush определяет цвет и стиль закрашки внутренней области многоугольника; параметр P (массив типа Point) — координаты углов многоугольника	g.FillPolygon(myBrush2, new PointF[] {new PointF(220, 20), new PointF(100, 100), new PointF(280, 100) });
DrawString(str, Font, Brush, x, y)	Выводит на графическую поверхность строку текста. Параметр Font определяет шрифт; Brush — цвет символов; x и y — точку, от которой будет выведен текст	Font currFont = new Font("Times New Roman", 24); g.DrawString("Тестирование шрифта...", currFont, new SolidBrush(Color.BlueViolet), 0, 0);
DrawImage(Image, x, y)	Выводит на графическую поверхность иллюстрацию. Параметр Image определяет иллюстрацию; x и y — координату левого верхнего угла области вывода иллюстрации	Image Bit1; Bit1 = new Bitmap("C:/C#/Image1/fl.bmp"); g.DrawImage(Bit1, 10, 10, 100, 100);

В качестве примера нарисуем светофор.

Пример 2.3.2. Светофор.

```
namespace My_Graph
{
    public partial class Form1 : Form
    {
        Graphics g; //Объявляем переменную g типа Graphics
        public Form1()
        {
            InitializeComponent();
            g = this.CreateGraphics(); //Создаем поверхность на форме
        }

        private void Form1_Paint(object sender, PaintEventArgs e)
        { //Рисуем корпус и окна светофора
            SolidBrush myBrushgray = new SolidBrush(Color.Gray);
            g.FillRectangle(myBrushgray, 40, 25, 200, 400);
            SolidBrush myBrushred = new SolidBrush(Color.Red);
            g.FillEllipse(myBrushred, 80, 25, 130, 130);
            SolidBrush myBrushYellow = new SolidBrush(Color.Yellow);
            g.FillEllipse(myBrushYellow, 80, 155, 130, 130);
            SolidBrush myBrushGreen = new SolidBrush(Color.Green);
            g.FillEllipse(myBrushGreen, 80, 285, 130, 130);
        }
    }
}
```

Задания для самостоятельной работы

Нарисовать:

1. Японский флаг.
2. Российский флаг.
3. Флаг Чехии.
4. Олимпийский флаг
5. Зимний пейзаж

2.3.4. Работа с изображениями

Класс *Image* имеет большое количество свойств и методов для работы с изображениями. Следующие свойства задают:

Width – ширину изображения

Height – высоту изображения

Size – размер изображения.

Объявленные переменные *Image* присваиваются объектам класса *Bitmap*. Кроме того, можно создавать объекты класса *Bitmap* напрямую. Например,

1. Объявим переменные типа *Image*:

```
private Image Bit1;  
private Image Bit2;  
private Image Bit3;
```

2. Создадим объекты класса *Bitmap*. Конструктор *Bitmap* возвращает объект типа *Image*.

```
Bit1 = new Bitmap("f1.bmp");  
Bit2 = new Bitmap("f2.bmp");
```

3. Выведем изображения на экран с помощью метода *DrawImage* и метода *FormPaint*.

```
e.DrawImage(Bit1, 10, 10, 100, 100);
```

Полностью код выглядит так:

```
private void Form1_Paint(object sender, PaintEventArgs e)  
{  
    Graphics p = e.Graphics;  
    Image Bit1;  
    Bit1 = new Bitmap("C:/C#/Image1/f1.bmp");  
    p.DrawImage(Bit1, 10, 10, 100, 100);  
}
```

2.4. Движение графических объектов

2.4.1. Компонент Timer

Для реализации движения мы будем использовать компонент *Timer*. Интервал времени таймера будем задавать с помощью компонента *NumericUpDown*.

Компонент *Timer* представляет собой компонент, который генерирует последовательность событий *Tick*. Другими словами, *Timer* – это цикл, который повторяется через заданный интервал времени. Компонент является невидимым. Свойства компонента приведены в табл. 8.

Таблица 8

Свойство	Описание
Interval	Период генерации события <i>Tick</i> . Задается в миллисекундах
Enabled	Разрешение работы. Разрешает (значение <i>True</i>) или запрещает (значение <i>False</i>) генерацию события <i>Tick</i>

2.4.2. Компонент *NumericUpDown*

Компонент NumericUpDown предназначен для ввода числовых данных. Данные в поле редактирования можно ввести путем набора на клавиатуре или при помощи командных кнопок Увеличить и Уменьшить, которые находятся справа от поля редактирования.

Свойства компонента NumericUpDown приведены в табл. 9.

Таблица 9

Свойство	Описание
Value	Значение, соответствующее содержимому поля редактирования
Maximum	Максимально возможное значение, которое можно ввести в поле компонента
Minimum	Минимально возможное значение, которое можно ввести в поле компонента
Increment	Величина, на которую увеличивается или уменьшается значение свойства Value при каждом щелчке мышью на кнопках Увеличить или Уменьшить

2.4.3. Неуправляемое движение графического объекта

На рисунке 2.14 представлено окно перед началом движение мяча. Кнопки start и stop запускают и останавливают таймер. Интервал таймера задает значение Value компонента NumericUpDown.

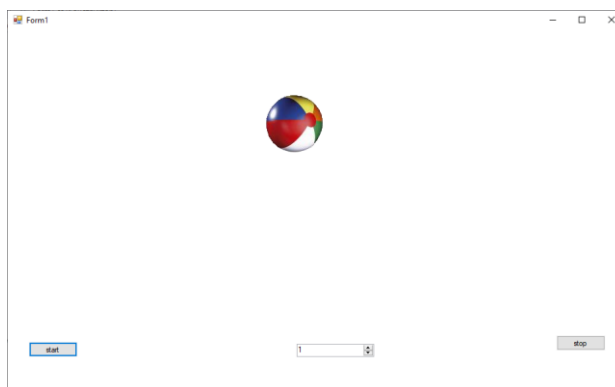


Рис. 2.14. Окно перед началом движения

Код программы с комментариями приведен ниже.

Пример 2.4.1. Неуправляемое движение

```
namespace Neupr_dv
{
    public partial class Form1 : Form
    {
        Graphics g; // Объявляем графическую поверхность
        Rectangle rct; // Прямоугольник с размерами рисунка
        private Image Ball; // Рисунок
        int x = 400, y = 100, dx = 2, dy = 2;

        private void button2_Click(object sender, EventArgs e)
        {
            timer1.Enabled = false; // Остановка таймера
        }

        private void timer1_Tick(object sender, EventArgs e)
        {
            rct.X = x; // Создаем прямоугольник
            rct.Y = y;
        }
    }
}
```

```

        rct.Width = Ball.Width;
        rct.Height = Ball.Height;
        Invalidate(rct); //Перерисовываем прямоугольник. Стираем мяч
        //Invalidate очищает прямоугольник. Если скобки пустые – всю форму.
        x += dx; //Меняем координаты мяча
        y += dy;
        g.DrawImage(Ball, x, y, Ball.Width, Ball.Height); //рисует мяч в новом
положении
        //Отражение от стенок
        if ((x < 0) || (x > (ClientRectangle.Width - Ball.Width))) dx = -dx;
        if ((y < 0) || (y > (ClientRectangle.Height - Ball.Height))) dy = -dy;
    }

    private void button1_Click(object sender, EventArgs e)
    {
        timer1.Interval = (int)numericUpDown1.Value * 10; //Интервал таймера
        timer1.Enabled = true; //Пуск таймера
    }

    private void Form1_Paint(object sender, PaintEventArgs e)
    {
        g.Clear(Color.White); //Окрашиваем форму
        g.DrawImage(Ball, x, y, Ball.Width, Ball.Height); //Рисуем мяч в начальном
положении
    }

    public Form1()
    {
        InitializeComponent();
        g = this.CreateGraphics(); //Создаем поверхность
        Ball = new Bitmap("D:/C#/My_Forms/beach_ball.png"); //Создаем рисунок мяча
    }
}

```

2.4.4. Движение объекта с помощью клавиш

Для реализации движения мы используем событие формы `Form1_KeyDown`. Обработчику события `private void Form1_KeyDown(object sender, KeyEventArgs e)` через параметр «e» передается код нажатой клавиши. Анализируя нажатия клавиш смещаем изображение, изменяя его начальные координаты.

Пример 2.4.2. Движение с помощью клавиш

```

namespace Upr_dv
{
    public partial class Form1 : Form
    {
        private Image Bit;
        int x = 300, y = 100, w = 40, h = 40;

        private void Form1_Paint(object sender, PaintEventArgs e)
        {
            g.Clear(Color.White);
            g.DrawImage(Bit, x, y, w, h);
        }

        private void Form1_KeyDown(object sender, KeyEventArgs e)
        {
            rct.X = x;
            rct.Y = y;
            rct.Width = w;
            rct.Height = h;
        }
    }
}

```

```

        Invalidate(rct);
        //Обработка кодов клавиш
        if (e.KeyCode == Keys.Down) y += 10;
        if (e.KeyCode == Keys.Up) y -= 10;
        if (e.KeyCode == Keys.Left) x -= 10;
        if (e.KeyCode == Keys.Right) x += 10;
        g.DrawImage(Bit, x, y, w, h);
    }

    Graphics g;
    Rectangle rct;
    public Form1()
    {
        InitializeComponent();
        g = this.CreateGraphics();
        Bit = new Bitmap("D:/C#/My_Projects/My_im/krest.png");
    }
}

```

2.4.5. Движение объекта с помощью мыши.

Событиям *MouseDown*, *MouseUp* и *MouseMove* в качестве аргумента передается объект *e*, который обеспечивает эти события текущими координатами (x,y) курсора мыши. В примере с помощью этих событий мы обеспечим движение объекта вслед за курсором мыши.

Пример 2.4.3. Движение с помощью мыши.

```

namespace My_im
{
    public partial class Form1 : Form
    {
        private Image Bit;
        int x = 10, y = 10, w = 40, h = 40;
        Graphics g;
        bool p = false; //Признак нажатой клавиши
        Rectangle rct;
        private void button1_Click(object sender, EventArgs e)
        {
            g = this.CreateGraphics();
            g.DrawImage(Bit, 100, 100, w, h);
        }

        private void Form1_MouseDown(object sender, MouseEventArgs e)
        {
            p = true; //Нажали
        }

        private void Form1_MouseUp(object sender, MouseEventArgs e)
        {
            p = false; //Отпустили
        }

        private void Form1_MouseMove(object sender, MouseEventArgs e)
        {
            if (p) //Если нажата, то
            {
                rct.X = x;
                rct.Y = y;
                rct.Width = w;
                rct.Height = h;
            }
        }
    }
}

```



```

        Invalidate(rct);
        x = e.X;
        y = e.Y;
        g.DrawImage(Bit, x, y, w, h);
    }
}

private void Form1_Paint(object sender, PaintEventArgs e)
{
    g.DrawImage(Bit, x, y, w, h);
}

public Form1()
{
    InitializeComponent();
    g = this.CreateGraphics();
    Bit = new Bitmap("D:/C#/My_Projects/My_im/krest.png");
}
}
}

```

2.4.6. Светофор

Представляем программу, демонстрирующую действующий светофор. На рис. 2.15. представлен светофор перед запуском.

Пример 2.4.4. Светофор.

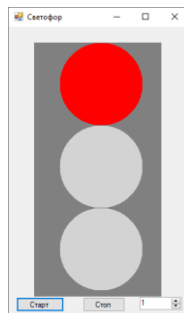


Рис. 2.15. Светофор

Код программы:

```

namespace My_Graph
{
    public partial class Form1 : Form
    {
        Graphics g; //Объявляем переменную g типа Graphics
        int nom =1;
        bool down = true;
        public Form1()
        {
            InitializeComponent();
            g = this.CreateGraphics(); //Создаем поверхность на форме
        }

        private void Form1_Paint(object sender, PaintEventArgs e)
        {
            SolidBrush myBrushgray = new SolidBrush(Color.Gray);
            g.FillRectangle(myBrushgray, 40, 25, 200, 400);
            SolidBrush myBrushred = new SolidBrush(Color.Red);
            g.FillEllipse(myBrushred, 80, 25, 130, 130);
        }
    }
}

```

```

        SolidBrush myBrushlg = new SolidBrush(Color.LightGray);
        g.FillEllipse(myBrushlg, 80, 155, 130, 130);
        g.FillEllipse(myBrushlg, 80, 285, 130, 130);
    }

    private void timer1_Tick(object sender, EventArgs e)
    {
        timer1.Interval = (int)numericUpDown1.Value * 100;
        switch (nom)
        {
            case 1:
            {
                SolidBrush myBrushred = new SolidBrush(Color.Red);
                g.FillEllipse(myBrushred, 80, 25, 130, 130);
                SolidBrush myBrushlg = new SolidBrush(Color.LightGray);
                g.FillEllipse(myBrushlg, 80, 155, 130, 130);
                down = true;
                nom = 2;
            }
            break;
            case 2:
            {
                SolidBrush myBrushYellow = new SolidBrush(Color.Yellow);
                g.FillEllipse(myBrushYellow, 80, 155, 130, 130);
                if (down)
                {
                    SolidBrush myBrushlg = new SolidBrush(Color.LightGray);
                    g.FillEllipse(myBrushlg, 80, 25, 130, 130);
                    nom = 3;
                }
                else
                {
                    SolidBrush myBrushlg = new SolidBrush(Color.LightGray);
                    g.FillEllipse(myBrushlg, 80, 285, 130, 130);
                    nom = 1;
                }
            }
            break;
            case 3:
            {
                SolidBrush myBrushGreen = new SolidBrush(Color.Green);
                g.FillEllipse(myBrushGreen, 80, 285, 130, 130);
                SolidBrush myBrushlg = new SolidBrush(Color.LightGray);
                g.FillEllipse(myBrushlg, 80, 155, 130, 130);
                down = false;
                nom = 2;
            }
            break;
        }
    }

    private void button1_Click(object sender, EventArgs e)
    {
        timer1.Enabled = true;
    }

    private void button2_Click(object sender, EventArgs e)
    {
        timer1.Enabled = false;
    }
}
}

```

2.4.7 Крестики-нолики

Еще один пример – код программы известной игры «Крестики-Нолики». Работа программы поясняется комментариями. На рис. 2.16 изображен момент игры.

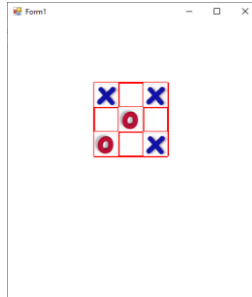


Рис. 2.16. Крестики-Нолики

Пример 2.4.5. Код программы

```
namespace Krest_No1
{
    public partial class Form1 : Form
    {
        int[,] Pole = new int[3, 3]; //Контрольный массив
        Graphics g;
        int nx = 140, ny = 100, dx = 40, dy = 40; //Координаты поля и размеры клетки
        int x, y;
        int klik = 1; //Переменная для очереди хода
        private Image Krest;
        private Image Nol;
        public Form1()
        {
            InitializeComponent();
            Pole = CrPole(3); //Метод заполняет контрольный массив
            g = this.CreateGraphics();
            Krest = new Bitmap("D:/C#/My_Projects/My_im/krest.png");
            Nol = new Bitmap("D:/C#/My_Projects/My_im/nol.png");
        }
        public void Drawpole() //Метод рисует поле
        {
            Pen myrPenRed = new Pen(Color.Red, 3);
            for (int i = 0; i < 4; i++)
                g.DrawLine(myrPenRed, nx, ny + i * dy, nx + 3 * dx, ny + i * dy);
            for (int i = 0; i < 4; i++)
                g.DrawLine(myrPenRed, nx + i * dx, ny, nx + i * dx, ny + 3 * dy);
        }
        private void Form1_Paint(object sender, PaintEventArgs e)
        {
            g.Clear(Color.White);
            Drawpole();
        }
        private void Form1_MouseDown(object sender, MouseEventArgs e)
        {
            int i, j, u, v;
            bool hor, ver, d1, d2;
            x = e.X;
            y = e.Y;
            //Если попали в игровое поле
```

```

if ((x > nx) && (x < nx + 3 * dx) && (y > ny) && (y < ny + 3 * dy))
{
    j = (x - nx) / dx; //Номер столбца и строки
    i = (y - ny) / dy;
    u = nx + j * dx; //Координаты угла клетки
    v = ny + i * dx;
    if (clik == 1) //Ход крестиков
    {
        Pole[i, j] = 1;
        g.DrawImage(Krest, u, v, dx, dy);
    }
    else //Ход ноликов
    {
        Pole[i, j] = -1;
        g.DrawImage(Nol, u, v, dx, dy);
    }
    //Проверяем выигрыш
    hor = Hor(i);
    ver = Ver(j);
    d1 = Diag1();
    d2 = Diag2();
    if ((hor) || (ver) || (d1) || (d2))
    { if (clik == 1)
      { MessageBox.Show("Krest"); }
      else
      { MessageBox.Show("Nol"); }
    }

    if (clik == 1) clik = 2;
    else clik = 1;
}

}

static int[,] CrPole(int n) //Заполняем поле нулями
{
    int[,] nums = new int[n,n];
    for (int i = 0; i < n; i++)
        for(int j=0;j<n;j++)
            nums[i,j] = 0;
    return nums;
}

public bool Hor(int i) //Проверка горизонтали
{
    bool res;
    int s = 0;
    for (int j = 0; j < 3; j++)
        s = s + Pole[i, j];
    s = Math.Abs(s);
    if (s == 3)
        res = true;
    else
        res = false;
    return res;
}

public bool Ver(int j) //Проверка вертикали
{
    bool res;
    int s = 0;
    for (int i = 0; i < 3; i++)
        s = s + Pole[i, j];
    s = Math.Abs(s);
    if (s == 3)

```

```

        res = true;
    else
        res = false;
    return res;
}
public bool Diag1()//Проверка главной диагонали
{
    bool res;
    int s = 0;
    s = Pole[1, 1] + Pole[2, 2] + Pole[0,0];
    s = Math.Abs(s);
    if (s == 3)
        res = true;
    else
        res = false;
    return res;
}
public bool Diag2()//Проверка второй диагонали
{
    bool res;
    int s = 0;
    s = Pole[0, 2] + Pole[1, 1] + Pole[2, 0];
    s = Math.Abs(s);
    if (s == 3)
        res = true;
    else
        res = false;
    return res;
}
}
}

```

2.5. Многооконные программы

Windows приложения, как правило, состоят из нескольких взаимодействующих между собой окон. В этом разделе мы познакомимся с техникой создания многооконных приложений в Window Forms.

Для того, чтобы добавить в проект форму, нужно войти в меню Проект и выбрать пункт Добавить форму. В выпадающем списке выбрать пункт Форма. После этого в проекте появляется новая форма.

Предположим, теперь у нас две формы. Но пока они никак не взаимодействуют и ничего друг про друга не знают. Объявим в коде первой формы вторую форму и дадим ей имя:

```

public partial class Form1 : Form
{
    public Form2 frm2;//Объявляем Form2 и даем ей имя frm2
}

```

Так же поступаем со второй формой:

```

public partial class Form2 : Form
{
    public Form1 frm1;
}

```

Положим на обе формы кнопку (button) и метку (label). По нашему замыслу мы нажатием кнопки в первой форме будем менять текст в метке второй формы и наоборот. Чтобы это стало возможным, мы должны у обеих меток установить свойство Modifiers public (по умолчанию – private).

Дальнейшие действия представлены в обработчиках кнопок первой и второй формы и комментариях к ним.

Первая форма:

```
private void button1_Click(object sender, EventArgs e)
{
    frm2 = new Form2(); //Создаем указатель на форму 2
    frm2.frm1 = this; //Передаем форме 2 указатель на форму 1
    frm2.BackColor = Color.Сyan; //Меняем цвет формы 2
    frm2.Show(); //Показываем форму 2
    frm2.label1.Text = "Привет"; //Меняем текст
    this.Hide(); //Прячем форму 1
}
```

Вторая форма:

```
private void button1_Click(object sender, EventArgs e)
{
    frm1.label1.Text = "Ответ";
    this.Hide(); //Прячем форму 2
    frm1.Show(); //Показываем форму 1
}
```

Важно: в определении функций и переменных, к которым мы хотим получить доступ, должен присутствовать модификатор доступа **public**, а для элементов управления свойство **Modifiers** должно быть установлено в **Public**.

На рис. 2.17 показаны первая форма, вторая форма и первая форма после взаимодействия со второй.

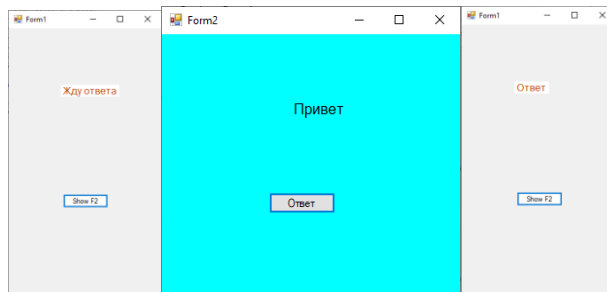


Рис.2.17. Взаимодействие форм.

2.6. Контейнеры в Window Forms.

Назначение контейнера понятно из его названия – хранение и группировка компонентов формы.

2.6.1. Контейнер GroupBox.

GroupBox представляет собой специальный контейнер, который ограничен от остальной формы границей. Он имеет заголовок, который устанавливается через свойство Text. Нередко этот элемент используется для группирования переключателей - элементов RadioButton, так как позволяет разграничить их группы.

Свойства этого контейнера приведены в табл. 10.

Таблица 10

Свойство	Описание
Text	Заголовок — текст, поясняющий назначение компонентов, которые находятся в поле компонента GroupBox
Enabled	Позволяет управлять доступом к компонентам, находящимся в поле (на поверхности) компонента GroupBox. Если значение свойства равно false, то все находящиеся в поле GroupBox компоненты недоступны
Visible	Позволяет скрыть (сделать невидимым) компонент GroupBox и все компоненты, которые находятся на его поверхности

2.6.2. Контейнер Panel

Компонент Panel представляет собой контейнер для других компонентов и позволяет легко управлять компонентами, которые находятся на панели. Свойства компонента Panel приведены в табл. 11.

Таблица 11

Свойство	Описание
Dock	Определяет границу формы, к которой привязана (прикреплена) панель. Панель может быть прикреплена к левой (Left), правой (Right), верхней (Top) или нижней (Bottom) границе формы
BorderStyle	Вид границы панели: FixedSingle — рамка; Fixed3D — объемная граница; None — граница не отображается
Enabled	Свойство позволяет сделать недоступными (false) все компоненты, которые находятся на панели
Visible	Позволяет скрыть (false) панель

2.7. Переключатели в Windows Forms

Переключатели в некотором смысле являются визуальными аналогами операторов if и switch.

2.7.1. Переключатель CheckBox.

Компонент CheckBox представляет собой переключатель, который может находиться в одном из двух состояний: выбранном или невыбранном. Свойства компонента CheckBox приведены в табл. 12.

Таблица 12

Свойство	Значение
Text	Текст, отображаемый справа от кнопки Checked
Checked	Состояние переключателя. Если переключатель выбран (в поле компонента отображается галочка), то значение свойства равно true. Если переключатель сброшен (галочка не отображается), то значение свойства равно false

TextAlign	Положение текста в поле отображения текста. Текст может располагаться в центре поля (MiddleCenter), быть прижат к левой (MiddleLeft) или правой (MiddleRight) границе. Можно задать и другие способы размещения текста надписи (TopLeft, TopCenter, TopRight, BottomLeft, BottomCenter, BottomRight)
CheckAlign	Положение кнопки в поле компонента. Кнопка может быть прижата к левой верхней границе (TopLeft), прижата к левой границе и находиться на равном расстоянии от верхней и нижней границ поля компонента (MiddleLeft). Есть и другие варианты размещения кнопки в поле компонента
Enabled	Управляет доступностью компонента. Позволяет сделать переключатель недоступным (false)
Visible	Управляет видимостью компонента. Позволяет скрыть, сделать невидимым (false) переключатель
AutoCheck	Определяет, должно ли автоматически изменяться состояние переключателя в результате щелчка на его изображении. По умолчанию значение равно true
FlatStyle	Стиль (вид) переключателя. Переключатель может быть обычным (Standard), плоским (Flat) или "всплывающим" (Popup). Стиль определяет поведение переключателя при позиционировании указателя мыши на его изображении
Appearance	Определяет вид переключателя. Переключатель может выглядеть обычным образом (Normal) или как кнопка (Button)
Image	Картинка, которая отображается в поле компонента. Положение картинки в поле компонента. Картинка может располагаться в центре (MiddleCenter), быть прижата к левой (MiddleLeft) или правой (MiddleRight) границе. Можно задать и другие способы размещения картинки на кнопке (TopLeft, TopCenter, TopRight, BottomLeft, BottomCenter, BottomRight)
ImageAlign	Положение картинки в поле компонента. Картинка может располагаться в центре (MiddleCenter), быть прижата к левой (MiddleLeft) или правой (MiddleRight) границе. Можно задать и другие способы размещения картинки на кнопке (TopLeft, TopCenter, TopRight, BottomLeft, BottomCenter, BottomRight)
ImageList	Набор картинок, используемых для обозначения различных состояний кнопки. Представляет собой объект типа ImageList. Чтобы задать значение свойства, в форму приложения следует добавить компонент ImageList
ImageIndex	Номер (индекс) картинки из набора ImageList, которая отображается в поле компонента

Приведем пример. В представленном на рис. 2.18 окне вы видите два переключателя, один из которых выбран. Кнопка меняет выбор переключателя и отображает текст выбранного переключателя в Label. Флажки помещены в контейнер `GroupBox`.

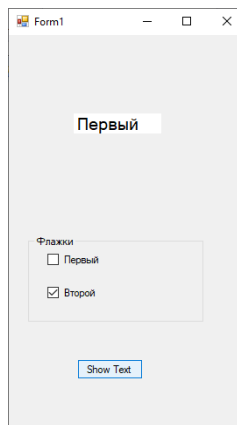


Рис. 2.18. Два переключателя

Код проекта:

Пример 2.7.1. Компоненты CheckBox.

```
public partial class Form1 : Form
{
    public Form1()
    {
        InitializeComponent();
    }

    private void button1_Click(object sender, EventArgs e)
    {
        if (checkBox1.Checked)
        {
            label1.Text = checkBox1.Text;
            checkBox1.Checked = false;
            checkBox2.Checked = true;
        }
        else
        {
            label1.Text = checkBox2.Text;
            checkBox2.Checked = false;
            checkBox1.Checked = true;
        }
    }
}
```

2.7.2. Переключатель RadioButton

Радиокнопка это зависимый переключатель, это означает, что из нескольких радиокнопок во включенном состоянии может находиться только одна кнопка. Радиокнопки, помещенные в контейнер работают, как группа зависимых переключателей, т.е. во включенном состоянии может находиться только одна кнопка. Контейнером для радиокнопок может служить форма, элементы GroupBox и Panel. Основные свойства RadioButton идентичны свойствам CheckBox, и вы можете видеть их в табл.12.

На рис. 2.19 представлена форма с тремя переключателями RadioButton. В зависимости от их выбора, кнопка переключает цвет метки.

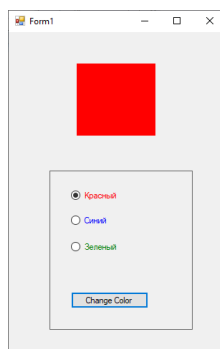


Рис. 19. Радиокнопки

Пример 2.7.2. Код программы

```
public partial class Form1 : Form
{
    public Form1()
    {
        InitializeComponent();
    }

    private void button1_Click(object sender, EventArgs e)
    {
        if (radioButton1.Checked) label1.BackColor = Color.Red;
        if (radioButton2.Checked) label1.BackColor = Color.Blue;
        if (radioButton3.Checked) label1.BackColor = Color.Green;
    }
}
```

2.8. Окно сообщений MessageBox.

Как правило, для вывода сообщений применяется элемент MessageBox. Однако кроме вывода строки сообщения данный элемент может устанавливать ряд настроек, которые определяют его поведение. Для вывода сообщения в классе MessageBox предусмотрен метод Show, который имеет различные версии и может принимать ряд параметров. Упрощенный синтаксис этого метода приведен ниже:

Show(<текст в окне>[,<заголовок окна>,<набор кнопок>,<иконка>, <выбранная по умолчанию кнопка >])

Рассмотрим более подробно параметры метода Show. Все параметры, кроме первого, являются необязательными. Первый и второй параметры метода строкового типа. Остальные параметры с помощью набора констант задают кнопки, иконки в окне и кнопку по умолчанию.

Тип MessageBoxButtons задает набор кнопок в окне. Набор констант типа MessageBoxButtons, приведен в табл. 13:

Таблица 13

Значение	Описание
AbortRetryIgnore	Окно содержит кнопки Abort, Retry и Ignore.
OK	Окно содержит кнопку ОК.
OKCancel	Окно содержит кнопки ОК и Cancel.
RetryCancel	Окно содержит кнопки Retry и Cancel.
YesNo	Окно содержит кнопки Yes и No.

YesNoCancel	Окно содержит кнопки Yes, No, и Cancel.
-------------	---

Тип `MessageBoxIcon` задает иконку в окне. Набор констант типа `MessageBoxIcon`, приведен в табл. 14:

Таблица 14

Значение	Описание
Asterisk	Сообщение содержит символ, состоящий из маленькой буквы i в кружке
Error	Пиктограмма ошибочного сообщения
Exclamation	Пиктограмма содержит восклицательный знак
Hand	Пиктограмма содержит символ, состоящий из белого крестика в кружке на красном фоне.
Information	Информационная пиктограмма
None	Ничего не содержит.
Question	Пиктограмма содержит вопросительный знак
Stop	Пиктограмма содержит символ, состоящий из белого крестика в кружке на красном фоне.
Warning	Пиктограмма содержит символ, состоящий из восклицательного знака в треугольнике на желтом фоне.

Перечисление `MessageBoxDefaultButton` определяет кнопку, выбранную по умолчанию. В таблице приведены члены перечисления:

Таблица 15

Значение	Описание
Button1	Первая кнопка в окне сообщения является кнопкой по умолчанию
Button2	Вторая кнопка в окне сообщения является кнопкой по умолчанию
Button3	Третья кнопка в окне сообщения является кнопкой по умолчанию

Метод `Show` возвращает значение типа `DialogResult`, который предоставляет набор констант, сигнализирующих какая из кнопок выбрана пользователем (см. табл. 16):

Таблица 16

Значение	Описание
Abort	Нажата кнопка <code>Abort</code> .
Cancel	Нажата кнопка <code>Cancel</code> .
Ignore	Нажата кнопка <code>Ignore</code> .
No	Нажата кнопка <code>No</code> .
None	Ничего не нажато.
OK	Нажата кнопка <code>OK</code> .
Retry	Нажата кнопка <code>Retry</code> .
Yes	Нажата кнопка <code>Yes</code> .

Покажем на примерах использование класса `MessageBox`.

Пример 2.8.1. Выдача сообщения в окне

```
MessageBox.Show("Текст", "Заголовок окна");
```

Вид окна представлен на рис. 2.20:

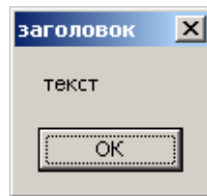


Рис. 2.20 Окно `MessageBox.Show("Текст", "Заголовок окна");`

Пример 2.8.2. Использование иконки в окне:

```
MessageBox.Show("Hello!", "Окно ввода", MessageBoxButtons.OK,
    MessageBoxIcon.Exclamation);
```

Вид окна на рис. 2.21:

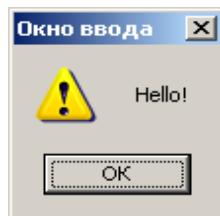


Рис. 2.21. Окно ввода

Пример 2.8.3. Использование нескольких кнопок

```
// удалить текущий элемент
if (MessageBox.Show("Вы уверены?", "Вопрос", MessageBoxButtons.OKCancel,
    MessageBoxIcon.Question) == DialogResult.OK)
    listBox1.Items.RemoveAt(listBox1.SelectedIndex);
```

Вид окна на рис. 2.22:

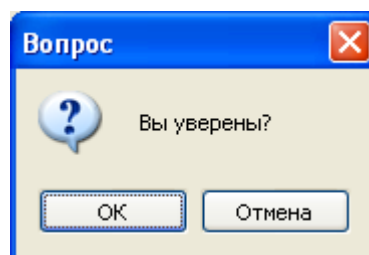


Рис. 2.22. Несколько кнопок

Пример 2.8.4. Кнопка «Отмена», выбранная по умолчанию:

```
if (MessageBox.Show("Вы уверены?", "Вопрос", MessageBoxButtons.OKCancel,
    MessageBoxIcon.Question,
    MessageBoxDefaultButton.Button2) == DialogResult.OK)
{
    MessageBox.Show("ok");
}
```

Вид окна на рис. 2.23:

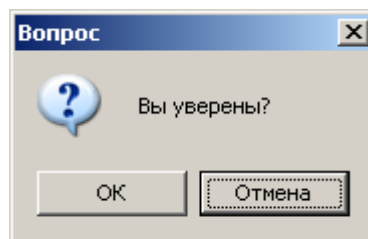


Рис. 2.23. Кнопка по умолчанию

2.9. Диалоги открытия и сохранения файлов

Окна открытия и сохранения файла представлены классами OpenFileDialog и SaveFileDialog. Они имеют во многом схожую функциональность, поэтому рассмотрим их вместе.

Элемент OpenFileDialog находится на вкладке «Диалоговые окна» панели инструментов и позволяет для выбора использовать стандартный Windows-диалог. Этот компонент является «невидимым». Его вызов осуществляется программным путем. Основные свойства диалога приведены в табл. 17

Таблица 17

Свойство	Описание
Title	Текст в заголовке окна. Если значение свойства не указано, то в заголовке отображается текст Открыть
Filter	Свойство задает описание и фильтр (маску) имени файла. В списке файлов отображаются только те файлы, имена которых соответствуют указанной маске. Описание отображается в поле Тип файла . Например, значение Текст *.txt указывает, что в списке файлов надо отобразить только те файлы, расширение которых txt
FilterIndex	Фильтр состоит из нескольких элементов (например, Текст *.txt Все файлы *.*), то значение свойства задает фильтр, который используется в момент появления диалога на экране
FileName	Имя выбранного пользователем файла
InitialDirectory	Каталог, содержимое которого отображается при появлении диалога на экране
RestoreDirectory	Признак необходимости отображать содержимое каталога, указанного в свойстве InitialDirectory, при каждом появлении окна. Если значение свойства равно False, то при следующем появлении окна отображается содержимое каталога

Элемент SaveFileDialog можно найти на вкладке «Диалоговые окна». OpenFileDialog и SaveFileDialog имеют ряд общих свойств. Они приведены в табл.17. В табл. 18 приведены свойства, специфические для SaveFileDialog.

Таблица 18

Свойство	Описание
----------	----------

CheckPathExists	Признак необходимости проверки существования каталога, в котором следует сохранить файл. Если указанного каталога нет, то выводится информационное сообщение
CheckFileExists	Признак необходимости проверки существования файла с заданным именем. Если значение свойства равно True и файл с указанным именем уже существует, то появляется окно запроса, в котором пользователь может подтвердить необходимость замены (перезаписи) существующего файла

Ниже приведен пример чтения и записи файла с использованием OpenFileDialog и SaveFileDialog.

Пример 2.9.1. Чтение из файла в TextBox и запись текста в файл.

```
private void button1_Click(object sender, EventArgs e)
{
    //Чтение из файла в TextBox
    if (openFileDialog1.ShowDialog() == DialogResult.OK)
    {
        string filename = openFileDialog1.FileName;
        string fileText = File.ReadAllText(filename);
        textBox1.Text = fileText;
    }
}

private void button2_Click(object sender, EventArgs e)
{
    //Запись из TextBox в файл
    saveFileDialog1.Filter = "Text files (*.txt)|*.txt|All files (*.*)|*.*";
    if (saveFileDialog1.ShowDialog() == DialogResult.Cancel)
        return;
    string filename = saveFileDialog1.FileName;
    File.WriteAllText(filename, textBox1.Text);
}
```

2.10. Списки ListBox и ComboBox.

2.10.1. Список ListBox.

Компонент `ListBox` представляет собой список, в котором можно выбрать нужный элемент. Ключевым свойством этого элемента является свойство `Items`, которое как раз и хранит набор всех элементов списка. Основные свойства компонента приведены в табл. 19.

Таблица 19

Свойство	Описание
<code>Items</code>	Элементы списка — коллекция строк
<code>Items.Count</code>	Количество элементов списка
<code>SelectedIndex</code>	Номер элемента, выбранного в списке. Если ни один из элементов списка не выбран, то значение свойства равно -1
<code>Sorted</code>	Признак необходимости автоматической сортировки (<code>True</code>) списка после добавления очередного элемента
<code>SelectionMode</code>	Определяет режим выбора элементов списка: <code>One</code> —

	только один элемент; <code>MultiSimple</code> — можно выбрать несколько элементов, сделав щелчок на нужных элементах списка; <code>MultiExtended</code> — можно выбрать несколько элементов, сделав щелчок на нужных элементах списка при нажатой клавише <code><Ctrl></code> , или выделить диапазон, щелкнув при нажатой клавише <code><Shift></code> на первом и последнем элементе диапазона
--	--

Элементы в список могут добавляться как во время разработки, так и программным способом. В Visual Studio в окне Properties (Свойства) для элемента `ListBox` мы можем найти свойство `Items`. После двойного щелчка на свойство нам отобразится окно для добавления элементов в список (рис.2.24):

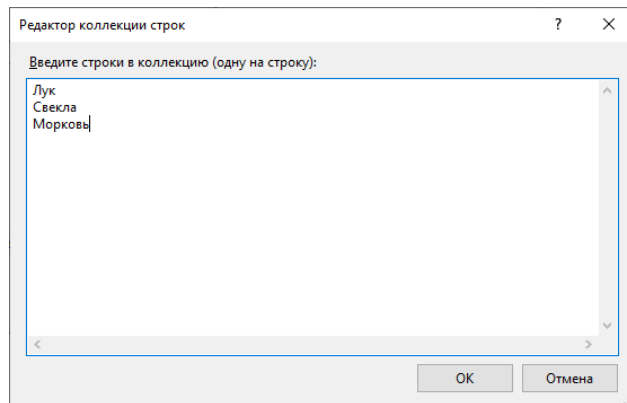


Рис.2.24. Окно добавления элементов.

В пустое поле мы вводим по одному элементу списка - по одному на каждой строке. После этого все добавленные нами элементы окажутся в списке, и мы сможем ими управлять (Рис 2.25).

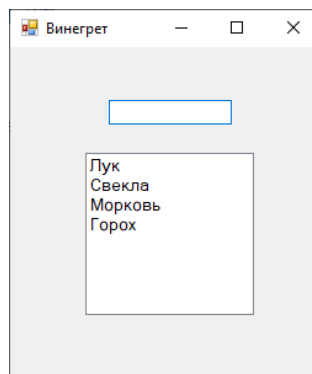


Рис. 2.25. `ListBox` с введенными элементами.

Все элементы списка входят в свойство `Items`, которое представляет собой коллекцию. Для добавления нового элемента в эту коллекцию, а значит и в список, надо использовать метод `Add`, например: `listBox1.Items.Add("Новый элемент");`. При использовании этого метода каждый добавляемый элемент добавляется в конец списка. Можно добавить сразу несколько элементов, например, массив. Для этого используется метод `AddRange`:

```
string[] countries = { "Бразилия", "Аргентина", "Чили", "Уругвай", "Колумбия" };
listBox1.Items.AddRange(countries)
```

В отличие от простого добавления вставка производится по определенному индексу списка с помощью метода Insert. В данном случае вставляем элемент на вторую позицию в списке, так как отсчет позиций начинается с нуля.

```
listBox1.Items.Insert(1, "Парагвай");
```

Для удаления элемента по его тексту используется метод Remove:

```
listBox1.Items.Remove("Чили");
```

Чтобы удалить элемент по его индексу в списке, используется метод RemoveAt:

```
listBox1.Items.RemoveAt(1);
```

Кроме того, можно очистить сразу весь список, применив метод Clear:

```
listBox1.Items.Clear();
```

Используя индекс элемента, можно найти сам элемент в списке. Например, получим первый элемент списка:

```
string firstElement = listBox1.Items[0];
```

Метод Count позволяет определить количество элементов в списке.

При выделении элементов списка мы можем ими управлять как через индекс, так и через сам выделенный элемент. Получить выделенные элементы можно с помощью следующих свойств элемента ListBox:

- **SelectedIndex:** возвращает или устанавливает номер выделенного элемента списка. Если выделенные элементы отсутствуют, тогда свойство имеет значение -1
- **SelectedIndices:** возвращает или устанавливает коллекцию выделенных элементов в виде набора их индексов
- **SelectedItem:** возвращает или устанавливает текст выделенного элемента
- **SelectedItems:** возвращает или устанавливает выделенные элементы в виде коллекции

По умолчанию список поддерживает выделение одного элемента. Чтобы добавить возможность выделения нескольких элементов, надо установить у его свойства SelectionMode значение MultiSimple.

Чтобы выделить элемент программно, надо применить метод SetSelected(int index, bool value), где index - номер выделенного элемента. Если второй параметр - value имеет значение true, то элемент по указанному индексу выделяется, если false, то выделение, наоборот, скрывается:

```
listBox1.SetSelected(2, true); // будет выделен третий элемент
```

Чтобы снять выделение со всех выделенных элементов, используется метод ClearSelected.

Рассмотрим пример. На рис. 2.26 представлена форма со списком ListBox и текстовый файл, в котором он сохранен.

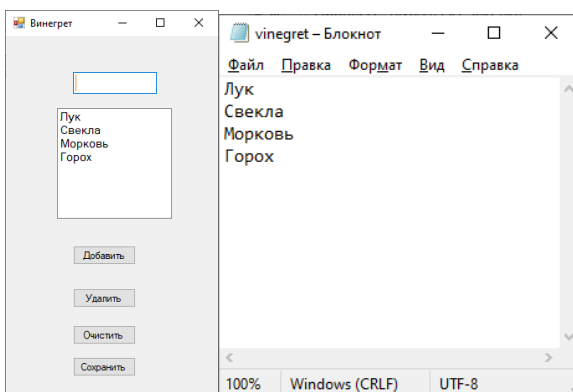


Рис. 2.26. ListBox и текстовый файл.

Ниже приведен код программы:

Пример 2.10.1. ListBox

```
private void button4_Click(object sender, EventArgs e)
{
    //Сохранение в файле
    saveFileDialog1.Filter = "Text files(*.txt)|*.txt|All files(*.*)|*.*";
    if (saveFileDialog1.ShowDialog() == DialogResult.Cancel)
        return;
    string filename = saveFileDialog1.FileName;
    WriteToFile(filename, listBox1);
}

private void WriteToFile(string path, ListBox listBox)
{
    using (var sw = new StreamWriter(new FileStream(path, FileMode.Create)))
    {
        if (listBox != null)
        {
            foreach (var item in listBox.Items) // в таком же порядке
            {
                sw.WriteLine(item.ToString());
            }
        }
    }
}

private void button1_Click(object sender, EventArgs e)
{
    //Добавляем элемент
    listBox1.Items.Add(textBox1.Text);
}

private void button2_Click(object sender, EventArgs e)
{
    //Удаляем элемент
    listBox1.Items.Remove("Горох");
}

private void button3_Click(object sender, EventArgs e)
{
    //Очищаем список
    listBox1.Items.Clear();
}
}
```

2.10.2. Список ComboBox.

Компонент ComboBox совмещает в себе список и текстовое поле. По этой причине у него есть свойство Text – текст, находящийся в поле ввода/редактирования. Отметим также свойство DropDownStyle – вид компонента: DropDown – поле ввода и раскрывающийся список; Simple – поле ввода со списком; DropDownList – раскрывающийся список. Ниже мы поясним действие этого свойства на примерах. Остальные его свойства аналогичны свойствам ListBox (см. табл. 19).

Для хранения элементов списка в ComboBox также предназначено свойство Items. Подобным образом, как и с ListBox, мы можем в окне свойств выбрать свойство Items и нам отобразится окно для добавления элементов ComboBox (рис. 2.27):

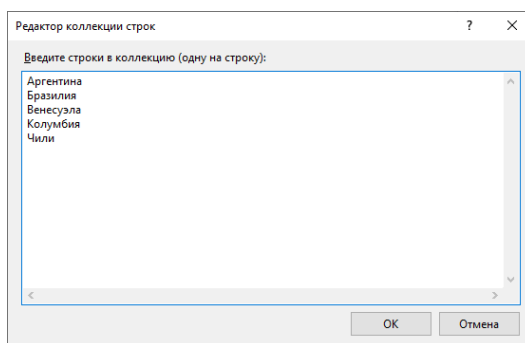


Рис. 2.27. Редактор строк ComboBox.

И, как и с компонентом ListBox, здесь мы также можем программно управлять элементами.

Добавление элементов:

```
// добавляем один элемент
comboBox1.Items.Add("Парагвай");
// добавляем набор элементов
comboBox1.Items.AddRange(new string[] { "Уругвай", "Эквадор" });
// добавляем один элемент на определенную позицию
comboBox1.Items.Insert(1, "Боливия");
```

Удаление элементов:

```
// удаляем один элемент
comboBox1.Items.Remove("Аргентина");
// удаляем элемент по индексу
comboBox1.Items.RemoveAt(1);
// удаляем все элементы
comboBox1.Items.Clear();
```

Мы можем получить элемент по индексу и производить с ним разные действия. Например, изменить его:

```
comboBox1.Items[0] = "Парагвай";
```

С помощью ряда свойств можно настроить стиль оформления компонента. Так, свойство `DropDownWidth` задает ширину выпадающего списка. С помощью свойства `DropDownHeight` можно установить высоту выпадающего списка.

Еще одно свойство `MaxDropDownItems` позволяет задать число видимых элементов списка - от 1 до 100. По умолчанию это число равно 8.

Другое свойство `DropDownStyle` задает стиль `ComboBox`. Оно может принимать три возможных значения:

- **DropDown:** используется по умолчанию. Мы можем открыть выпадающий список вариантов при вводе значения в текстовое поле или нажав на кнопку со стрелкой в правой части элемента, и нам отобразится собственно выпадающий список, в котором можно выбрать возможный вариант
- **DropDownList:** чтобы открыть выпадающий список, надо нажать на кнопку со стрелкой в правой стороне элемента
- **Simple:** `ComboBox` представляет простое текстовое поле, в котором для перехода между элементами мы можем использовать клавиши клавиатуры вверх/вниз

На рис. 2.28 Представлены три варианта оформления `ComboBox`:

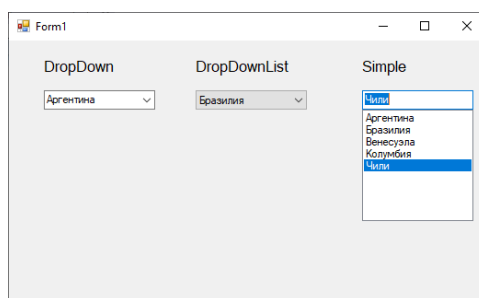


Рис.2.28. Варианты оформления `ComboBox`

2.11. Главное меню. Элемент `MenuStrip`.

Для создания меню в Windows Forms применяется элемент `MenuStrip`. После создания проекта следует разместить его на форме.

Для создания меню в визуальном режиме предназначен *конструктор меню*, который активизируется при выделении компонента `menuStrip1`. В более ранних, чем версия 2019, версиях Microsoft Visual Studio для создания главного меню приложения используется встроенный построитель меню – специальное окно. В версии 2019 поле ввода с текстом `Type Here` появляется на месте будущего пункта меню. В него следует ввести имя создаваемого пункта меню. После ввода имени и нажатия клавиши `<Enter>` в меню будет создан новый пункт, а рядом и под ним будут выведены новые поля ввода с текстом `Type Here`, позволяющие создать очередные пункты меню первого или второго уровня. При выделении созданного пункта меню в окне `Properties` отображаются свойства данного пункта.

Имя пункта меню, в отличие от имен других компонентов, размещаемых в форме, получается не добавлением порядкового номера к имени типа (например, `label1`), а приписыванием имени команды меню слева к имени типа, например, `fileToolStripMenuItem` (это правило действует даже для пунктов меню с русскими названиями, например, `файлToolStripMenuItem`). В результате получаются очень длинные имена, поэтому сразу после создания пункта меню желательно изменить имя пункта, указав новое значение его

свойства Name. Заметим, что свойство Name располагается в окне Properties в *начале* списка свойств; причем подпись к нему заключается в скобки: (Name).

На событие Click элемента меню пишется программный код, который будет выполняться при нажатии на пункт меню.

На рис. 2.29 представлена форма с пунктами меню, переключающими цвета Label.

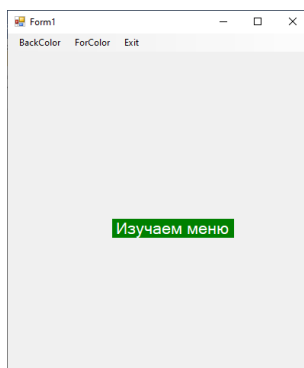


Рис. 2.29. Форма с меню.

Программный код приведен в следующем примере:

Пример 2.11.1. Разработка меню.

```
//Обработчики пунктов меню
private void green_Click(object sender, EventArgs e)
{
    label1.BackColor = Color.Green;
}

private void blue_Click(object sender, EventArgs e)
{
    label1.BackColor = Color.Blue;
}

private void white_Click(object sender, EventArgs e)
{
    label1.ForeColor = Color.White;
}

private void yellow_Click(object sender, EventArgs e)
{
    label1.ForeColor = Color.Yellow;
}

private void magenta_Click(object sender, EventArgs e)
{
    label1.ForeColor = Color.Magenta;
}

private void exit_Click(object sender, EventArgs e)
{
    //Закрываем проект
    this.Close();
}
```

2.12. Учебная викторина Миллионер

Телевизионная игра «Кто хочет стать миллионером?» хорошо известна. Наша викторина создана на ее основе, поэтому пояснять ее правила мы не будем. Это упражнение направлено на развитие у учащихся навыков работы с рядом компонентов Windows Forms.

В начале на основном окне видны только пункты меню и приглашение начать игру (рис. 2.30):

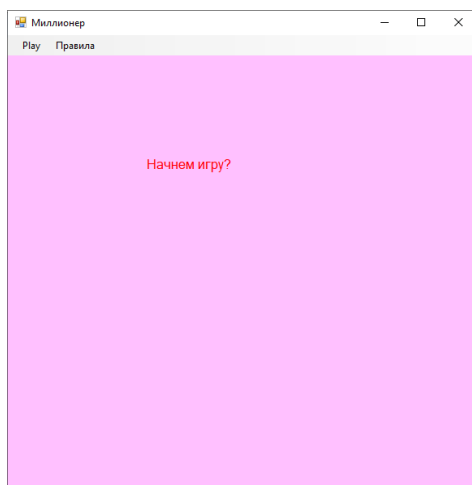


Рис. 2.30. Вид окна до начала игры

Выбирая пункт меню «Play», мы начинаем игру. Появляется ListBox с вариантами ответа и сам вопрос в Label (рис. 2.31):

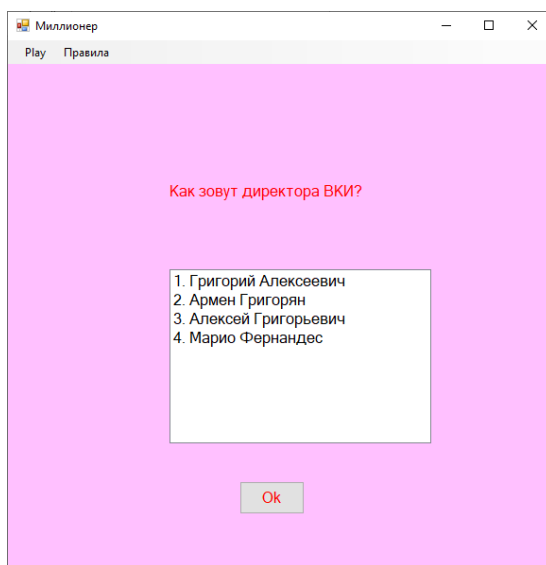


Рис. 2.31. Окно в процессе игры

Пунктом меню «Правила» мы вызываем второе окно с текстом помощи (рис. 2.32):

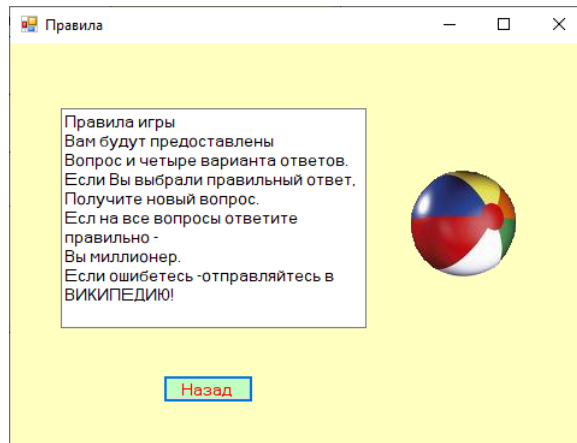


Рис. 2.32. Окно помощи

Пример 2.12.1. Код викторины миллионер с комментариями.

Код первой формы:

```
public partial class Form1 : Form
{
    public Form2 frm2;
    List<string> ask = new List<string>(); //Объявление списка вопросов
    int otvet;
    public Form1()
    {
        InitializeComponent();
    }

    private void play_Click(object sender, EventArgs e)
    {
        label1.Visible = true; //Управление игрой видимо
        listBox1.Visible = true;
        button1.Visible = true;
        ask.AddRange(File.ReadAllLines(@"D:\C#\My_Text\questions.txt"));
        label1.Text = ask[0]; //Вопрос
        for (int i = 1; i < 5; i++) //ответы
            listBox1.Items.Add(ask[i]);
        otvet = Convert.ToInt32(ask[5]); //Правильный ответ
    }

    private void правила_Click(object sender, EventArgs e)
    {
        frm2 = new Form2(); //Создаем указатель на форму 2
        frm2.frm1 = this; //Передаем форме 2 указатель на форму 1
        frm2.Show(); //Показываем форму 2
        this.Hide(); //Прячем форму 1
    }

    private void button1_Click(object sender, EventArgs e)
    {
        if (listBox1.SelectedIndex == otvet)
        {
            MessageBox.Show("Ok");
            ask.RemoveRange(0, 6); //Удаляем предыдущий набор
            label1.Text = ask[0]; //Новый вопрос
            listBox1.Items.Clear();
            for (int i = 1; i < 5; i++)

```

```

        listBox1.Items.Add(ask[i]);
        otvet = Convert.ToInt32(ask[5]);
    }
    else
    {
        MessageBox.Show("Почитай Википедию");
        label1.Visible = false; //Управление невидимо
        listBox1.Visible = false;
        button1.Visible = false;
    }
}
}
}

```

Код второй формы:

```

public partial class Form2 : Form
{
    public Form1 frm1;
    public Form2()
    {
        InitializeComponent();
    }

    private void button1_Click(object sender, EventArgs e)
    {
        this.Hide(); //Прячем форму 2
        frm1.Show(); //Показываем форму 1
    }
}

```

2.13. Создание компонентов программным способом

В разделе 2.2. мы обсуждали создание компонентов формы с помощью панели управления. Но зачастую есть необходимость создавать компоненты по ходу выполнения программы. Еще один случай: бывает необходимо создать массив компонентов и работать с ним с помощью цикла. Рассмотрим эти вопросы. Начнем с программного создания кнопки.

Кнопка будет создана при нажатии на заготовленную с помощью панели управления кнопку. Код этой программы представлен в примере. На рис. 2.33 представлено окно с программно-созданной кнопкой Mb.

Пример 2.13.1. Код создания кнопки и обработчик события созданной кнопки.

```

private void button1_Click(object sender, EventArgs e)
{
    Button MyButton = new Button(); //Создаем кнопку с именем MyButton
    MyButton.Location = new Point(65, 25); //Задаем ее положение на форме
    MyButton.Text = "Mb"; //Текст
    MyButton.BackColor = Color.White; //Цвет фона
    MyButton.Size = new Size(70, 40); //Размер
    MyButton.Font = new Font("Arial", 10, FontStyle.Regular); //Шрифт
    MyButton.ForeColor = Color.Red; //Цвет текста
    //Создаем указатель на метод кнопки. Объект (делегат) EventHandler()
    //может указывать на любую функцию
    MyButton.Click += b1_Click;
    //Добавляем кнопку в коллекцию элементов формы
    Controls.Add(MyButton);
}

```

```
//Обработчик события кнопки MyButton
public void b1_Click(object sender, EventArgs e)
{
    //sender-объект, на котором произошло событие.
    //Преобразуем его в кнопку
    Button Tb = sender as Button;
    // Button Tb = (Button)sender; Можно преобразовать и так
    MessageBox.Show(Tb.Text);
}
}
```

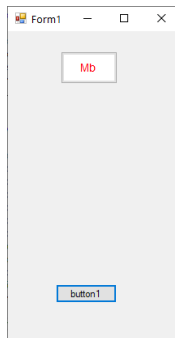


Рис. 2.33. Программно-созданная кнопка

Теперь создадим массив кнопок. Конкретно: создадим четыре кнопки, пронумеруем их и дадим каждой имя. Программа загадывает случайное число, а играющий должен угадать, какое это число. Для этого он мышью кликает по одной из кнопок. Данный проект – простейший пример динамического создания кнопок и их методов. Форма проекта после начала игры приведена на рис. 2.34.

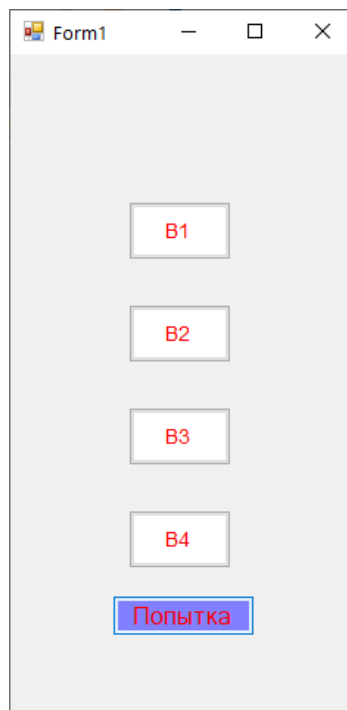


Рис. 2.34. Динамически созданный массив кнопок

Ниже приведен код программы с комментариями:

Пример 2.13.2. Код проекта динамического создания массива кнопок

```
namespace Lohotron
{
    public partial class Form1 : Form
    {
        Button[] b1 = new Button[4]; //Массив кнопок
        int l; //Случайное число
        public Form1()
        {
            InitializeComponent();
        }
        public void MYButton()
        { //Формируем массив кнопок
            for (int i = 0; i < 4; i++)
            {
                b1[i] = new Button();
                b1[i].Left = 80;
                b1[i].Text = "В" + (i + 1).ToString();
                b1[i].Top = 100 + i * 70;
                b1[i].Size = new Size(70, 40);
                b1[i].Tag = i;
                b1[i].Click += b1_Click;
                b1[i].BackColor = Color.White;
                b1[i].ForeColor = Color.Red;
                b1[i].Font = new Font("Arial", 10, FontStyle.Regular);
                b1[i].Hide(); //Прячем кнопки массива
                Controls.Add(b1[i]);
            }
        }

        private void Form1_Load(object sender, EventArgs e)
        {
            MYButton(); //Вызываем MYButton
        }
        public void b1_Click(object sender, EventArgs e)
        {
            Button Tb;
            Tb = (Button)sender; //Преобразование
            if ((int)Tb.Tag == 1) //Преобразуем Tag в int
            {
                MessageBox.Show("Победа");
                for (int i = 0; i < 4; i++)
                    b1[i].Hide(); //Прячем все кнопки
            }
            else
                Tb.Hide(); //Прячем кнопку
        }

        private void button1_Click(object sender, EventArgs e)
        {
            Random rnd = new Random();
            l = rnd.Next(0, 3); //Загадали число
            for (int i = 0; i < 4; i++)
                b1[i].Show(); //Показали кнопки
        }
    }
}
```

В качестве примера динамического создания двумерного массива кнопок приведем разработку известной настольной игры «Пятнанадцать». Ее игровое поле представлено на рис.2.35.

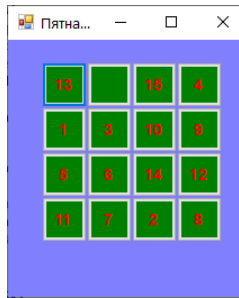


Рис 2.35. Игра «Пятнадцать».

Пример 2.13.2. Код программы игра «Пятнадцать».

```
public partial class Form1 : Form
{
    //Контрольный массив
    public int[,] a = new int[4, 4];
    int h = 40; //Размер кнопки
    int x0 = 30; //Угол игрового поля
    int y0 = 20;
    int i0, k0;
    Button[,] bt = new Button[4, 4]; //Массив кнопок
    public Form1()
    {
        InitializeComponent();
    }
    //Перестановка
    public void swap(ref int a, ref int b)
    {
        int r;
        r = b;
        b = a;
        a = r;
    }
    //Заполнение контрольного массива
    public void zapolnenie()
    {
        int i, j, k, m;
        bool zap;
        int n = 4;
        Random rnd = new Random();
        for (i = 0; i < 4; i++)
            for (j = 0; j < 4; j++)
                a[i, j] = 0;
        for (i = 0; i < 15; i++)
        {
            zap = true;
            while (zap)
            {
                k = rnd.Next(0, 4);
                m = rnd.Next(0, 4);
                if (a[k, m] == 0)
                {
                    a[k, m] = i + 1;
                    zap = false;
                }
            }
        }
    }
}
```

```

private void Form1_Load(object sender, EventArgs e)
{
    заполнение();
    int i,j;
    for (i=0;i<4;i++)
        for (j=0; j<4; j++)
        {
            bt[i,j] = new Button();
            bt[i,j].Left = x0+j*h;
            bt[i,j].Top = y0+i*h;
            bt[i, j].Width = h;
            bt[i, j].Height = h;
            bt[i,j].Font= new Font("Arial", 10, FontStyle.Bold);
            bt[i, j].BackColor = Color.Green;
            bt[i, j].ForeColor = Color.Red;
            if (a[i, j] != 0)
                bt[i, j].Text = a[i, j].ToString();
            else
            { //Пустое поле
                i0 = i;
                k0 = j;
            }
            bt[i,j].Click += bt_Click;
            Controls.Add(bt[i,j]);
        }
    }
    public void bt_Click(object sender, EventArgs e)
    {
        int i,k;
        Button Tb;
        Tb = (Button)sender;
        k = (Tb.Left - x0)/h;
        i = (Tb.Top - y0)/ h;
        //Проверка наличия рядом пустого поля
        if (((Math.Abs(i-i0)==1)&&(k==k0))||((Math.Abs(k - k0)==1)&&(i==i0)))
        {
            swap(ref a[i,k],ref a[i0, k0]);
            bt[i, k].Text = "";
            bt[i0, k0].Text = a[i0, k0].ToString();
            //Новое пустое поле
            i0 = i;
            k0 = k;
        }
    }
}

```

2.14. Компонент PictureBox.

Компонент **PictureBox** обеспечивает отображение иллюстрации (рисунка, фотографии и т. п.). Свойства компонента приведены в табл. 20

Таблица 20.

Свойство	Описание
Image	Иллюстрация, которая отображается в поле компонента
SizeMode	Режим отображения иллюстрации (способ масштабирования), если размер иллюстрации не соответствует размеру компонента:

	Normal — масштабирование не выполняется (если размер компонента меньше размера иллюстрации, то отображается только часть иллюстрации); StretchImage — выполняется масштабирование иллюстрации таким образом, что она занимает всю область отображения (если размер компонента не пропорционален размеру иллюстрации, она будет искажена); AutoSize — размер компонента автоматически изменяется и соответствует размеру иллюстрации; CenterImage — центрирование иллюстрации в поле компонента, если размер иллюстрации меньше размера области отображения
Image.PhysicalDimension	Свойство содержит информацию о размере картинки (иллюстрации), загруженной в поле компонента
Location	Положение компонента (области отображения иллюстрации) на поверхности формы. Уточняющее свойство X определяет расстояние от левой границы области до левой границы формы, уточняющее свойство Y — от верхней границы области до верхней границы клиентской области формы (нижней границы заголовка)
Size	Размер компонента (области отображения иллюстрации). Width определяет ширину области, Height — высоту
BorderStyle	Вид границы компонента: None — граница не отображается; FixedSingle — тонкая; Fixed3D — объемная
Graphics	Поверхность, на которую можно выводить графику (доступ к графической поверхности компонента есть у процедуры обработки события Paint)

В качестве примера использования компонента PictureBox создадим известную настольную игру Puzzle – сборку картинки из ее фрагментов. Выведем на форму рисунок. Затем создадим двумерный массив компонентов PictureBox. С помощью свойства `Pict.Clone(new Rectangle(m * w, k * h, w, h), Pict.PixelFormat)`, копирующего фрагменты рисунка разместим эти фрагменты в элементах массива. С помощью датчика случайных чисел перемешаем фрагменты. Обработчик события `OnClick` будет осуществлять перестановку фрагментов изображения. В качестве «переменной» для перестановки используем невидимый PictureBox. Игровое поле с перемешанной картинкой приведено на рис. 2.36.

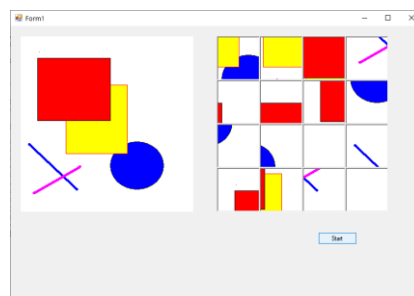


Рис. 2.36. Игровое поле “Puzzle”.

Ниже приведен код программы с комментариями

Пример 2.14.1. Код программы Puzzle/

```
namespace Puzzle
{
    public partial class Form1 : Form
    {
        Bitmap Pict; //Переменная для рисунка
        Graphics g; //Графическая поверхность для рисунка
        PictureBox[,] Pb = new PictureBox[4, 4]; //Массив PictureBox
        int h, w, i0, j0;
        int klik = 1; //Счет кликов
        public Form1()
        {
            InitializeComponent();
            g = this.CreateGraphics(); //Создаем поверхность
        }

        private void button1_Click(object sender, EventArgs e)
        { //Обработчик кнопки Start
            int i, j, k, m;
            bool zap;
            int[,] a = new int[4, 4]; //Массив используется для перемешивания
            Image bb;
            int x0 = 400;
            int y0 = 20;
            Random rnd = new Random();
            // Если Pb[i,j] есть на форме, мы его удаляем
            for (i = 0; i < 4; i++)
                for (j = 0; j < 4; j++)
                    if (this.Controls.Contains(Pb[i, j]))
                        this.Controls.Remove(Pb[i, j]);
            for (i = 0; i < 4; i++)
                for (j = 0; j < 4; j++)
                    a[i, j] = 0; //Заполняем нулями
            for (i = 0; i < 4; i++)
                for (j = 0; j < 4; j++)
                { //Создаем массив
                    Pb[i, j] = new PictureBox();
                    Pb[i, j].Left = x0 + j * w;
                    Pb[i, j].Top = y0 + i * h;
                    Pb[i, j].Width = w;
                    Pb[i, j].Height = h;
                    //Граница должна быть видна
                    Pb[i, j].BorderStyle = BorderStyle.Fixed3D;
                    Pb[i, j].Click += Pb_Click;
                    Controls.Add(Pb[i, j]);
                    zap = true;
                    while (zap)
                    {
                        k = rnd.Next(0, 4);
                        m = rnd.Next(0, 4);
                        if (a[k, m] == 0) //Если в элементе массива Pb нет рисунка
                        {
                            a[k, m] = 1;
                            zap = false;
                            //Здесь мы копируем из рисунка фрагмент
                            //Копирование идет не с рисунка на форме, а из рисунка в памяти.
                            //Поэтому координаты левого верхнего угла - 0,0.
                            bb = Pict.Clone(new Rectangle(m * w, k * h, w, h),
                                Pict.PixelFormat);
                            Pb[i, j].Image = bb; //Вставляем фрагмент и Pb[i,j]
                        }
                    }
                }
        }
    }
}
```

```

    }
}
public void Pb_Click(object sender, EventArgs e)
{
    //Обработчик щелчка по Pb[i,j]
    int x0 = 400;
    int y0 = 20;
    PictureBox Pib;
    Pib = (PictureBox)sender;
    if (clik == 1)
    {
        j0 = (Pib.Left - x0) / w; //Номер столбца и строки элемента,
        i0 = (Pib.Top - y0) / h; //по которому кликнули - запомнили.
        pictureBox1.Image = Pib.Image; //Изображение сохранили
        clik = 2;
    }
    else
    {
        //Перестановка
        Pb[i0, j0].Image = Pib.Image;
        Pib.Image = pictureBox1.Image;
        clik = 1;
    }
}

private void Form1_Paint(object sender, PaintEventArgs e)
{
    //Рисуем картинку
    Pict = new Bitmap("D:/C#/My_Text/puzzle.bmp");
    g.DrawImage(Pict, 20, 20, Pict.Width, Pict.Height);
    //Размеры фрагмента рисунка
    w = Pict.Width / 4;
    h = Pict.Height / 4;
    //Невидимый pictureBox1 для перестановки фрагментов
    pictureBox1.Width = w;
    pictureBox1.Height = h;
    pictureBox1.Visible = false;
}
}
}
}

```

Заключение

Windows Forms предоставляет программисту большие возможности и для создания интерфейса, и для разработки логических игр. В одном методическом пособии невозможно раскрыть весь потенциал Windows Forms. Но, как нам кажется, мы наметили для студентов и преподавателей пути для дальнейшего, более глубокого исследования визуального программирования на языке C#. Желаем вам успехов на этом пути!

Литература

1. Иванчева Н.А., Иваньчева Т.А. Программирование на языке C# в среде Visual Studio Net с примерами задач. Новосибирск, ВКИ НГУ, 2016.
2. [Создание проекта приложения Windows Forms - Visual Studio | Microsoft Docs](#)
3. Культин Н. Б. C# в задачах и примерах. — СПб.: БХВ-Петербург, 2007. — 240 с.
4. Абрамян М. Э. Visual C# на примерах. — СПб.: БХВ-Петербург, 2008. — 496 с.
5. [Руководство по программированию в Windows Forms \(metanit.com\)](#)
6. Учебник по языку C# на платформе .NetCore3. <https://metanit.com/sharp/tutorial/>. 2020.